CREAERA UI · DOKUMENTATION

Das Handbuch

Komponenten, Design-System und moderne Angular-Architektur

Mit NgRx Signal Store, HTTP-APIs und Angular-/Nx-Befehlen.

Export vom 25. Oktober 2026
Repository-Stand: ce81ebc · main

Offline lesbare Fassung der Storybook-Anwender- und Architektur-Dokumentation. Enthält die textlichen Anleitungen und Codebeispiele; interaktive Stories und automatisch generierte Komponenten-Controls sind in Storybook verfügbar.

Inhaltsverzeichnis

1. Anwenderdoku
2. Farben zentral anpassen
3. Light und Dark Mode
4. Responsive Größen
5. Komponenten verwenden
6. Tree und Tree Table
7. Formularfelder
8. Autocomplete
9. API und Projektaufbau
10. Architektur – Einstieg
11. Module und Nx
12. Komponenten anlegen und CLI
13. Komponenten und APIs
14. State und Signal Store
15. HTTP und CRUD
16. Aktionen und Nebenläufigkeit
17. Server-Suche
18. Tests und Weiterentwicklung

Creaera UI verwenden

Die Komponentenbibliothek stellt Angular-Komponenten mit einer bewusst kleinen API bereit. Gemeinsame Farben und Größen werden zentral gepflegt. Es gibt keine Farb- oder Größenparameter pro Komponente.

Orientierung in Storybook

- **Anwenderdoku:** Einrichtung, zentrale Farben, Light/Dark Mode, responsive Größen und die vollständige API-/Projektreferenz.
- **Foundations → Overview:** alle Komponenten auf einer gemeinsamen Beispielseite.
- **Components:** einzelne Komponenten, Varianten, Controls und automatisch erzeugte API-Dokumentation unter **Docs**.

Die Toolbar in den Komponenten-Stories bietet **Farbschema** (`light`, `dark`) und **Größenprofil** (`auto`, `small`, `large`). Zum Ausprobieren eine Story oder **Foundations → Overview** öffnen. Die Textseiten dieser Anwenderdoku verwenden Storybooks eigene Dokumentationsoberfläche; die Toolbar verändert die Komponenten-Vorschau, nicht das Theme der Storybook-Navigation.

Lokal starten

Voraussetzungen: Node.js 24 ab Version 24.15 und npm.

```
npm ci
npm run storybook
```

Storybook startet auf Port 6006. In einer Cloud-Umgebung benötigt der Zugriff von außen eine Portfreigabe. Es sind keine Backend-Dienste oder Zugangsdaten nötig.

Komponenten in Angular einbinden

Alle Komponenten und Direktiven sind standalone und aus `@creaera/ui` importierbar. Das Alias ist im Nx-Workspace bereits eingerichtet.

```
import { Component } from '@angular/core';
import { Button, Checkbox } from '@creaera/ui';

@Component({
  selector: 'app-example',
  imports: [Button, Checkbox],
  template: `
    <button uiButton (click)="save()">Speichern</button>
    <ui-checkbox [(checked)]="enabled">Benachrichtigungen</ui-checkbox>
  `,
```

```
})
export class Example {
  enabled = false;

  save() {
    // Aktion der Anwendung
  }
}
```

Angular-Bindings wie `[(checked)]` stehen im Angular-Template. Die API aller Komponenten und weitere Beispiele finden sich unter **API und Projektaufbau**.

Globale Styles einbinden

Im Workspace `libs/shared/ui/src/styles.css` in die globalen Styles der Angular-Anwendung aufnehmen. Storybook macht das bereits über seine Projektkonfiguration. Die Datei lädt Tailwind CSS und `theme.css`.

Beispiel für den `styles`-Eintrag eines Angular-Build-Targets in `project.json`:

```
{
  "styles": ["libs/shared/ui/src/styles.css", "apps/my-app/src/styles.css"]
}
```

Die anwendungseigene CSS-Datei kommt danach, damit dort Theme-Tokens überschrieben werden können. Den Beispielpfad `apps/my-app` an das eigene Projekt anpassen. Die PostCSS-Konfiguration des Workspaces enthält bereits `@tailwindcss/postcss`.

Das gebaute Paket exportiert `@creaera/ui/styles.css` und `@creaera/ui/theme.css`. In einem anderen Tailwind-Projekt wird die erste Datei über dessen Tailwind-/PostCSS-Pipeline importiert. Sie ist Tailwind-Quell-CSS und muss verarbeitet werden. Wer fertiges CSS ohne Tailwind-Pipeline benötigt, kann es im Workspace erzeugen und danach global laden:

```
npx @tailwindcss/cli -i libs/shared/ui/src/styles.css -o dist/ui.css
```

Eine Veröffentlichung in einer Paket-Registry ist damit nicht verbunden.

Theme und Größenprofil setzen

```
<body class="ui-dark ui-large">
  <!-- Angular-Anwendung -->
</body>
```

Ohne Klassen gelten Light Mode und die automatische Größenwahl nach Bildschirmbreite. Es sollte immer nur eine Theme-Klasse und höchstens eine Größenklasse aktiv sein. Beide Einstellungen sind voneinander unabhängig.

Bauen und prüfen

Befehl	Ergebnis
`npm run build`	Paketierbare Angular-Bibliothek in `dist/libs/shared/ui`
`npm run storybook`	Interaktiver Entwicklungsserver auf Port 6006
`npm run build-storybook`	Statisches Storybook inklusive Anwenderdoku in `dist/storybook`
`npm test`	Komponententests einschließlich Forms und Tastaturbedienung
`npm run test:e2e`	Storybook-Build und echte Browsertests
`npm run format:check`	Formatierungsprüfung

Für die Browsertests einmalig `npx playwright install chromium` ausführen oder `PLAYWRIGHT_CHROMIUM_EXECUTABLE_PATH` auf einen vorhandenen Chromium setzen. Cards stehen als normale und ausklappbare Variante bereit. Formularfelder und ein Angular-Forms-Beispiel stehen ebenfalls bereit.

Farben zentral konfigurieren

Die zentrale Konfiguration ist **`libs/shared/ui/src/theme.css`**. Sie enthält CSS-Variablen für alle semantischen Farbrollen und beide Größenprofile. `styles.css` verwendet diese Variablen für die Komponenten und ordnet ausgewählte Tokens Tailwind-Utilities zu. Die Ausgangspalette ist Grün.

Aufbau der Farbkonfiguration

```
:root,
.ui-light {
  /* Farbwerte für Light Mode */
  --ui-interaction-primary: #146c43;
}

.ui-dark {
  /* Farbwerte für Dark Mode */
  --ui-interaction-primary: #237a4f;
}
```

Zum Ändern der Bibliotheksvorgaben die Werte direkt in `theme.css` bearbeiten. Die Anpassung gilt für alle Komponenten. Für eine eigene Anwendung können alternativ Tokens in deren globaler CSS-Datei **nach** den Bibliotheksstyles überschrieben werden. Dafür sind keine Änderungen an Komponenten nötig.

Alle Farbrollen

Bereich	CSS-Variable	Verwendung / Bedeutung
Background	`--ui-background-primary`	Seitenhintergrund
Background	`--ui-background-secondary`	Flächen und Menü-Hintergründe
Background	`--ui-background-tertiary`	Graue Flächen, etwa Tertiärbuttons
Fonts	`--ui-font-active`	Schrift auf gefüllten Buttons
Fonts	`--ui-font-default`	Standardtext und Textbuttons
Fonts	`--ui-font-disabled`	Text auf deaktivierten gefüllten Buttons
Interactions	`--ui-interaction-primary`	Primäraktionen und ausgewählte Controls
Interactions	`--ui-interaction-secondary`	Sekundäraktionen
Interactions	`--ui-interaction-disabled`	Deaktivierte Interaktionsflächen bzw. Texte

Bereich	CSS-Variable	Verwendung / Bedeutung
Interactions	`--ui-interaction-hover`	Hover-Farbe gefüllter Primär-/Tertiärbuttons
Interactions	`--ui-interaction-pressed-primary`	Gedrückte Primäraktionen
Interactions	`--ui-interaction-pressed-secondary`	Gedrückte Sekundäraktionen
Interactions	`--ui-interaction-divider`	Trennlinien und dezente Umrandungen
Status	`--ui-status-normal`	Normaler Status
Status	`--ui-status-caution`	Hinweis / Vorsicht
Status	`--ui-status-dangerous`	Gefahr / Fehler

Statusfarben stehen als Design-Tokens bereit. Eigenständige Statuskomponenten sind noch nicht Bestandteil der Bibliothek.

Card-Flächen sind bewusst in beiden Themes weiß. Die zusätzlichen zentralen Tokens `--ui-card-background` und `--ui-card-text` steuern ihre Fläche und den dunklen Text. `--ui-interaction-hover-secondary` steuert den grauen Hover-Zustand der Sekundärbuttons. Die Sekundärfarbe und deren Pressed-Farbe sind ebenfalls grau.

Beispiel: eine andere Grünpalette

In der globalen Anwendungs-CSS-Datei nach den Bibliotheksstyles:

```
:root,
.ui-light {
  --ui-interaction-primary: #166534;
  --ui-interaction-secondary: #52525b;
  --ui-interaction-hover: #14532d;
  --ui-interaction-pressed-primary: #104326;
  --ui-interaction-pressed-secondary: #3f3f46;
  --ui-background-primary: #f0fdf4;
  --ui-font-default: #153523;
}

.ui-dark {
  --ui-interaction-primary: #28734c;
  --ui-interaction-secondary: #60606b;
  --ui-interaction-hover: #205f3f;
  --ui-interaction-pressed-primary: #174b31;
  --ui-interaction-pressed-secondary: #45454f;
  --ui-background-primary: #0d1b13;
  --ui-font-default: #e4f4e9;
}
```

Beide Themes separat pflegen und bei Änderungen auch Hover, Pressed, Disabled und Focus prüfen. Die Beispiele sind Startwerte, keine verbindliche Farbpalette.

Tailwind-Utilities

Die Zuordnung steht in `libs/shared/ui/src/styles.css` unter `@theme inline`. Beispiele:

```
<section class="bg-ui-background text-ui-text p-ui-padding">
  <p class="text-ui-normal">Gespeichert</p>
  <p class="text-ui-caution">Bitte prüfen</p>
  <p class="text-ui-dangerous">Fehler</p>
</section>
```

Utility-Beispiel	Zugeordnetes Token
`bg-ui-background`	`--ui-background-primary`
`bg-ui-surface`	`--ui-background-secondary`
`bg-ui-primary`	`--ui-interaction-primary`
`bg-ui-secondary`	`--ui-interaction-secondary`
`text-ui-text`	`--ui-font-default`
`border-ui-divider`	`--ui-interaction-divider`
`text-ui-normal`	`--ui-status-normal`
`text-ui-caution`	`--ui-status-caution`
`text-ui-dangerous`	`--ui-status-dangerous`

Tailwind muss die Templates der konsumierenden Anwendung als Quellen erfassen. Falls die automatische Erkennung dort nicht greift, diese über eine eigene `@source`-Angabe ergänzen.

Bewusst feste Farben

Die weißen Button-Borders, weißen Icons und weißen Switch-Knöpfe sind derzeit entsprechend den Anforderungen fest weiß und keine separaten Farb-Tokens. Checkboxen und Radios nutzen zusätzlich native Browserdarstellung mit `accent-color`. Daher lassen sich nicht sämtliche Details ihrer Darstellung über die Palette verändern.

Light und Dark Mode

Das Theme wird zentral über eine Klasse am `body` ausgewählt. Die Komponenten erben die CSS-Variablen und wechseln gemeinsam ihr Farbschema. Ein Theme-Input an jeder Komponente ist nicht erforderlich.

Light Mode

```
<body class="ui-light">
  <!-- Angular-Anwendung -->
</body>
```

Ohne Theme-Klasse gilt ebenfalls Light Mode, weil die Light-Tokens auf `:root` liegen.

Dark Mode

```
<body class="ui-dark">
  <!-- Angular-Anwendung -->
</body>
```

`.ui-dark` überschreibt die Farb-Tokens und setzt `color-scheme: dark`, damit auch unterstützte native Browser-Controls das dunkle Schema berücksichtigen. Es wird nicht automatisch anhand der Betriebssystemeinstellung umgeschaltet.

Zur Laufzeit wechseln

Die bisherige Theme-Klasse entfernen, bevor die neue gesetzt wird:

```
function setTheme(theme: 'light' | 'dark'): void {
  document.body.classList.remove('ui-light', 'ui-dark');
  document.body.classList.add(`ui-${theme}`);
}

setTheme('dark');
```

Das Beispiel ist für den Browser gedacht. Falls die Anwendung serverseitig rendert, darf der Zugriff auf `document` nur im Browser erfolgen. Speichern der Nutzerpräferenz oder Übernehmen von `prefers-color-scheme` ist Aufgabe der konsumierenden Anwendung; die Bibliothek legt dafür keine zusätzliche API fest.

Eigene Flächen und Texte

Damit auch selbst erstellte Layouts umschalten, dort die Tokens oder die zugehörigen Tailwind-Utilities nutzen. Ein Theme ändert nicht automatisch jeden beliebigen Hintergrund der Anwendung:

```
body {
  background: var(--ui-background-primary);
  color: var(--ui-font-default);
}
```

```
<main class="bg-ui-background text-ui-text">
  <!-- Eigener Inhalt und UI-Komponenten -->
</main>
```

Theme für einen Teilbaum

Die Klassen können auch auf einem Container liegen. Das ist etwa für eine dunkle Vorschau innerhalb einer hellen Anwendung sinnvoll:

```
<section class="ui-dark bg-ui-background text-ui-text p-ui-padding">
  <button uiButton>Aktion im dunklen Bereich</button>
</section>
```

Auf einen Container nicht gleichzeitig `ui-light` und `ui-dark` setzen. Bei verschachtelten Containern bestimmt der nächste Container mit Theme-Klasse die vererbten Tokens.

Mit Größenprofilen kombinieren

Theme und Größe sind unabhängig:

```
<body class="ui-dark ui-small"></body>
<body class="ui-light ui-large"></body>
```

Die beiden Zeilen sind alternative Einstellungen, keine zwei Body-Elemente auf einer Seite. Ohne Größenklasse bleibt die automatische Größenwahl erhalten.

In Storybook ausprobieren

1. **Foundations** → **Overview** oder eine einzelne Komponenten-Story öffnen.
2. In der Toolbar **Farbschema** öffnen.
3. `light` oder `dark` auswählen.
4. Hover, gedrückten Zustand, Tastaturfokus und deaktivierte Varianten prüfen.

Die Toolbar verändert die Komponenten-Vorschau. Storybooks eigene Sidebar und Dokumentationsoberfläche haben ihre eigene Darstellung.

Responsive Größen

Die Bibliothek besitzt zwei Größenprofile. Die Bildschirmbreite wählt das Profil automatisch; eine zentrale Klasse kann diese Wahl überschreiben. Das Theme bleibt davon unabhängig.

Automatische Wahl

Ohne `ui-small` oder `ui-large` gilt:

Viewportbreite	Profil
Unter `48rem`	Klein
Ab einschließlich `48rem`	Groß

`48rem` entspricht bei der üblichen Ausgangsschriftgröße von 16px ungefähr 768px. Maßgeblich ist die Viewportbreite, nicht die Breite eines einzelnen Containers. Das ist ein gemeinsamer Breakpoint, keine Erkennung eines bestimmten Gerätetyps.

```
<body class="ui-light">
  <!-- Größenprofil wechselt automatisch mit dem Viewport -->
</body>
```

Profil ausdrücklich vorgeben

```
<body class="ui-light ui-small"></body>
<body class="ui-dark ui-large"></body>
```

Dies sind alternative Einstellungen. Immer höchstens eine Größenklasse setzen. `ui-small` erzwingt das kleine Profil auch auf einem großen Bildschirm; `ui-large` erzwingt das große Profil auch auf einem kleinen Bildschirm. Die Klassen können auch auf einem Container einen Teilbaum betreffen.

Zurück zur automatischen Auswahl:

```
document.body.classList.remove('ui-small', 'ui-large');
```

Zur Laufzeit ein bestimmtes Profil setzen:

```
function setSize(size: 'auto' | 'small' | 'large'): void {
  document.body.classList.remove('ui-small', 'ui-large');
  if (size !== 'auto') document.body.classList.add(`ui-${size}`);
}
```

Zentrale Größen-Tokens

Die Werte stehen in **`libs/shared/ui/src/theme.css`**:

Token	Klein	Groß	Verwendung
`--ui-padding`	`0.75rem`	`1rem`	Gemeinsames Padding, unter anderem Menüeinträge
`--ui-margin`	`0.75rem`	`1rem`	Gemeinsamer Margin-Token für eigene Layouts
`--ui-button-height`	`2.75rem`	`3rem`	Mindesthöhe und Mindestbreite der Buttons
`--ui-button-padding`	`1rem`	`1.25rem`	Horizontaler Button-Innenabstand
`--ui-font-size`	`0.875rem`	`1rem`	Schriftgröße der Buttons und Auswahl-Labels
`--ui-control-size`	`1.25rem`	`1.375rem`	Größe nativer Checkboxen und Radios
`--ui-radius`	`0.625rem`	`0.625rem`	Gemeinsamer Radius, derzeit ohne Profilunterschied

Nicht jede Komponente verwendet jeden Token. Chips und der Switch haben derzeit zusätzliche feste Maße in `styles.css`; sie werden nicht vollständig durch das Button-Höhen-Token skaliert. Es gibt keinen lokalen Größen-Input pro Komponente.

Werte ändern

`theme.css` hat vier relevante Bereiche:

1. `:root`: Standardwerte für das kleine Profil sowie gemeinsame Werte.
2. `@media (min-width: 48rem)`: automatische Werte für das große Profil.
3. `.ui-small`: erzwungene kleine Werte.
4. `.ui-large`: erzwungene große Werte.

Bei einer Profiländerung **die automatische Variante und die entsprechende Klassenvariante gemeinsam** ändern. Andernfalls weichen automatische und manuell gewählte Größe voneinander ab.

Beispiel: Buttonhöhe für das große Profil auf `3.25rem` anheben:

```
@media (min-width: 48rem) {
  :root {
    --ui-button-height: 3.25rem;
  }
}
```

```
.ui-large {
  --ui-button-height: 3.25rem;
}
```

Diese Regeln entweder in der zentralen Datei anpassen oder nach den Bibliotheksstyles in der Anwendung ergänzen. Für einen anderen Breakpoint die Media Query in der zentralen Datei ändern. Die Breakpoint-Grenze ist CSS-Syntax und kein zur Laufzeit überschreibbarer CSS-Variablenwert.

Padding und Margin in eigenen Layouts

```
<section class="p-ui-padding m-ui-margin">
  <!-- Layout nutzt dieselben zentralen Abstände -->
</section>
```

Alternativ direkt CSS verwenden:

```
.my-section {
  padding: var(--ui-padding);
  margin: var(--ui-margin);
}
```

Der Margin-Token verteilt Abstände nicht automatisch auf jedes Element. Die Anwendung entscheidet, wo sie ihn verwendet.

In Storybook prüfen

1. **Foundations** → **Overview** oder eine Komponenten-Story öffnen.
2. In der Toolbar **Größenprofil** auf `auto`, `small` oder `large` setzen.
3. Für `auto` die Viewportbreite über und unter `48rem` ändern.
4. Prüfen, dass `small` bzw. `large` auch bei anderer Viewportbreite bestehen bleibt.

Die Toolbar setzt die Klassen auf den Body der Vorschau, nicht auf die Storybook-Sidebar. Sie verändert außerdem nicht selbst die Bildschirmauflösung.

Komponenten verwenden

Alle folgenden Beispiele sind **Angular-Templates**. Die jeweils verwendeten Standalone-Exporte aus `@creaera/ui` im `imports`-Array der eigenen Komponente registrieren. Die vollständige Input-/Output-Tabelle steht unter **API und Projektaufbau**.

Buttons und Icons

`Button` importieren. Native Attribute und Events bleiben nutzbar:

```
<button uiButton (click)="save()">Speichern</button>
<button uiButton variant="secondary">Zurück</button>
<button uiButton variant="tertiary">Weitere Aktion</button>
<button uiButton variant="text">Abbrechen</button>
<button uiButton [disabled]="busy">Speichern</button>
<button uiButton type="submit">Formular absenden</button>
```

Ohne `type` gilt `button`, sodass ein Button ein umgebendes Formular nicht versehentlich absendet. Die Varianten heißen `primary` (Standard), `secondary`, `tertiary` und `text`.

Zustand	Verhalten
Default	Primär-/Sekundärfarbe; Tertiär grau mit Primärborder; Text transparent
Hover	Weiße Border; gefüllte Buttons verwenden die gemeinsame Hover-Farbe
Pressed	Dickere weiße Innenumrandung und Pressed-Farbe; Text bleibt transparent
Disabled	Weiße Border, Disabled-Farbe, keine native Aktivierung
Focused	Doppelter weißer Fokusrahmen mit grünem Hintergrund für Sichtbarkeit

Hover, Pressed und Focused entstehen durch echte Maus-/Tastaturbedienung. Sie werden nicht über Inputs gesetzt. Icons werden projiziert und sind weiß:

```
<button uiButton>
  <svg uiIcon aria-hidden="true" viewBox="0 0 24 24">
    <path d="M12 5v14M5 12h14" stroke="currentColor" stroke-width="2" />
  </svg>
  Hinzufügen
</button>

<button uiButton aria-label="Hinzufügen">
  <svg uiIcon aria-hidden="true" viewBox="0 0 24 24">
```

```
    <path d="M12 5v14M5 12h14" stroke="currentColor" stroke-width="2" />
  </svg>
</button>
```

Beim Icon-only Button ist `aria-label` nötig. Dekorative Icons bekommen `aria-hidden="true"`. `uiIcon` ist ein Styling-Attribut und benötigt keinen zusätzlichen Angular-Import.

Checkbox und Switch

`Checkbox` bzw. `ToggleSwitch` importieren. Der projizierte Text ist das Label. Mit `[(checked)]` wird der Zustand in beide Richtungen gebunden:

```
<ui-checkbox [(checked)]="accepted">Bedingungen akzeptieren</ui-checkbox>
<ui-switch [(checked)]="autosave">Automatisch speichern</ui-switch>
<ui-checkbox [disabled]="true">Nicht verfügbar</ui-checkbox>
<ui-switch [checked]="true" [disabled]="true">Aktiv, aber gesperrt</ui-switch>
```

Die zugehörigen Felder `accepted` und `autosave` sind Booleans. „Active“ bei der Checkbox meint den bedienbaren Zustand; `checked` steuert die Auswahl. Beim Switch entspricht `checked` dem aktiven/eingeschalteten Zustand.

Angular Forms

Checkbox und Switch unterstützen Angular Forms ohne zusätzlichen Adapter. Beispiel mit Reactive Forms:

```
import { Component } from '@angular/core';
import { FormControl, ReactiveFormsModule } from '@angular/forms';
import { Checkbox, ToggleSwitch } from '@creaera/ui';

@Component({
  selector: 'app-settings',
  imports: [ReactiveFormsModule, Checkbox, ToggleSwitch],
  template: `
    <ui-checkbox [formControl]="notifications">Benachrichtigungen</ui-checkbox>
    <ui-switch [formControl]="autosave">Automatisch speichern</ui-switch>
  `,
})
export class Settings {
  notifications = new FormControl(false, { nonNullable: true });
  autosave = new FormControl(true, { nonNullable: true });
}
```

Mit `control.disable()` / `control.enable()` den Formularzustand steuern. Alternativ `FormsModule` importieren und `[(ngModel)]` verwenden; innerhalb eines Formulars außerdem einen eindeutigen `name` setzen. Pro Control entweder Forms oder `[(checked)]` als Zustandsquelle wählen.

Radio Buttons

`Radio` und bei Verwendung von `ngModel` zusätzlich `FormsModule` importieren:

```
<fieldset>
  <legend>Versandart</legend>
  <label class="ui-choice">
    <input type="radio" uiRadio name="delivery" value="standard" [(ngModel)]="delivery" />
    Standard
  </label>
  <label class="ui-choice">
    <input type="radio" uiRadio name="delivery" value="express" [(ngModel)]="delivery" />
    Express
  </label>
  <label class="ui-choice">
    <input type="radio" uiRadio name="delivery" value="pickup" disabled />
    Abholung
  </label>
</fieldset>
```

Das Feld `delivery` kann zum Beispiel mit `'standard'` initialisiert werden. `type="radio"` muss ausdrücklich im Template stehen, damit Angular den Radio-Forms-Accessor auswählt. Gleicher `name` bedeutet gemeinsame Auswahlgruppe; verschiedene Gruppen erhalten unterschiedliche Namen. `[formControl]` ist ebenfalls möglich. `fieldset` und `legend` geben der Gruppe eine Beschriftung.

Button Group

`ButtonGroup` und für die Typisierung `UiOption` importieren. Optionen haben eindeutige Werte:

```
import type { UiOption } from '@creaera/ui';

// Felder der konsumierenden Angular-Komponente:
const options: UiOption[] = [
  { value: 'day', label: 'Tag' },
  { value: 'week', label: 'Woche' },
  { value: 'month', label: 'Monat', disabled: true },
];
```

Einfachauswahl:

```
<ui-button-group label="Ansicht" [options]="options" [(selected)]="view" />
```

`view` ist ein `readonly string[]`, zum Beispiel `['day']`. Auch bei Einfachauswahl wird ein Array mit höchstens einem Eintrag verwendet. Bei einem leeren Array ist zunächst nichts ausgewählt. In der Einfachauswahl wechseln Pfeiltasten sowie Home/End die Auswahl und überspringen gesperrte Optionen.

Mehrfachauswahl:

```
<ui-button-group
  label="Sichtbare Zeiträume"
  [options]="options"
  [multiple]="true"
  [(selected)]="periods"
/>
```

`periods` ist zum Beispiel `['day', 'week']`. Ein weiterer Klick auf einen aktiven Button entfernt ihn aus der Auswahl. `[disabled]="true"` sperrt die gesamte Gruppe. `label` ist für eine zugängliche Gruppenbeschriftung erforderlich. Die Gruppe unterstützt kein `formControl`-/`ngModel`-Binding; sie verwendet `[(selected)]`.

Chips

`Chip` importieren. Ein Chip enthält einen kleinen X-Button mit dem zugänglichen Namen „Entfernen: …“, ergänzt um seinen sichtbaren Text:

```
@if (showTag) {
<ui-chip (removed)="showTag = false">Designsystem</ui-chip>
}
<ui-chip disabled>Nicht verfügbar</ui-chip>
```

`removed` meldet den Entfernungswunsch. Die Anwendung entfernt den Eintrag aus ihrer Liste; der Chip verändert die Daten nicht selbst. Der X-Button funktioniert mit Maus, Enter und Leertaste und sendet bei Disabled kein Event. Er löst auch keine Klickaktion eines umgebenden Containers aus.

Die bisherige Schreibweise `<button uiChip>` wird durch `<ui-chip>` ersetzt, damit keine Buttons ineinander liegen. Der Chip ist kein ausgewählter Filter. Default, Hover, Pressed, Disabled und Fokus werden über CSS dargestellt.

Cards

`Card` importieren. Eine normale Card zeigt ihren Inhalt immer an:

```
<ui-card>
  <h2 uiCardHeader>Projektdetails</h2>
  <p>Zusammengehörige Informationen.</p>
</ui-card>
```

Mit `uiCardHeader` wird der persistente Kopf markiert. Alle anderen Inhalte liegen im Inhaltsbereich. Das Attribut benötigt keinen eigenen Angular-Import. Die Anwendung wählt eine zur Überschriftenhierarchie passende Ebene (h2, h3 usw.).

Ausklappbare Variante

```
<ui-card collapsible label="Projektdetails" [(expanded)]="detailsOpen">
  <h2 uiCardHeader>Projektdetails</h2>
  <p>Zusätzliche Informationen im ausklappbaren Inhaltsbereich.</p>
</ui-card>
```

`detailsOpen` ist ein Boolean. Standardmäßig ist eine ausklappbare Card geschlossen. Der Header bleibt sichtbar; der Pfeil unten mittig öffnet/schließt den Inhalt. Enter und Leertaste funktionieren wie bei einem nativen Button. `label` benennt die Card für den zugänglichen Toggle-Text; der Standardwert ist „Card“.

Zwei zusätzliche, rein dekorative Ebenen erzeugen die Optik von drei gestapelten Cards. Sie erscheinen nur bei der ausklappbaren Variante. Die Pfeilrichtung und `aria-expanded` folgen dem Zustand. Der Inhalt wird ausgeblendet, nicht zerstört; Zustände von Kindkomponenten bleiben beim Einklappen erhalten.

Sekundärborder und Verschachtelung

```
<ui-card>
  <h2 uiCardHeader>Projektübersicht</h2>
  <ui-card collapsible border="secondary" label="Weitere Details">
    <h3 uiCardHeader>Weitere Details</h3>
    <p>Diese Card lässt sich unabhängig öffnen.</p>
  </ui-card>
</ui-card>
```

`border="primary"` ist der Standard. `border="secondary"` verwendet die graue Sekundärfarbe. Cards können auch innerhalb ausklappbarer Cards liegen. Jede Card hat ihren eigenen Zustand und eine eigene ID für den kontrollierten Bereich.

Die Flächen bleiben auch im Dark Mode weiß, mit dunklem Text. Die zentralen Tokens heißen `--ui-card-background` und `--ui-card-text`. Padding und Abstände folgen `--ui-padding` und `--ui-margin`; die Rundung folgt `--ui-radius`.

Dropdown-Menü

`Dropdown` importieren. Die Optionen verwenden ebenfalls `UiOption`:

```
<ui-dropdown label="Aktionen" [options]="actions" (selected)="handleAction($event)" />
```

Das Feld `actions` ist ein Optionsarray, etwa `[{ value: 'copy', label: 'Duplizieren' }]`. `handleAction(value: string)` erhält den `value` der aktivierten Option. Einzelne Optionen lassen sich mit `disabled: true` sperren, das ganze Menü mit `[disabled]="true"`.

Das Dropdown ist ein **Aktionsmenü**, kein Formular-Select. Es speichert keine dauerhafte Auswahl und unterstützt kein `formControl`-/`ngModel`-Binding.

Option Button / Split Button

`OptionButton` importieren. Der Hauptbutton führt eine Aktion aus; der separate Pfeil öffnet weitere Aktionen:

```
<ui-option-button
  label="Speichern"
  [options]="saveOptions"
  (action)="save()"
  (selected)="handleSaveOption($event)"
/>
```

`saveOptions` ist ein `UiOption[]`. `(action)` hat keinen Nutzwert; `(selected)` liefert den String-Wert der gewählten Menüoption. Ein Klick auf den Pfeil löst die Hauptaktion nicht aus. `[disabled]="true"` sperrt beide Buttons.

Menübedienung mit Tastatur

Taste / Aktion	Verhalten
Enter oder Leertaste auf dem Trigger	Menü öffnen bzw. schließen
Pfeil abwärts auf dem Trigger	Öffnen und ersten verfügbaren Eintrag fokussieren
Pfeil aufwärts auf dem Trigger	Öffnen und letzten verfügbaren Eintrag fokussieren
Pfeil aufwärts / abwärts im Menü	Zwischen verfügbaren Einträgen navigieren
Home / End	Ersten / letzten verfügbaren Eintrag fokussieren
Enter oder Leertaste auf einem Eintrag	Aktion auslösen und Menü schließen
Escape	Schließen und Fokus zum Menütrigger zurückgeben
Tab oder Klick außerhalb	Menü schließen

Die sichtbaren Beschriftungen und `label`-Inputs sind Teil der zugänglichen Bedienung. Deshalb sinnvolle Namen wählen, auch wenn Beispiele kurze Texte verwenden.

Tree und Tree Table

Beide Komponenten stellen hierarchische Daten dar. **Tree** zeigt die Baumstruktur, **Tree Table** ergänzt Datenspalten. Zum Start verwenden beide Einfachauswahl und einen gemeinsamen Zustand für offene Knoten. Die API bleibt bewusst klein.

Datenmodell

```
import type { UiTreeNode, UiTreeColumn } from '@creaera/ui';

const nodes: UiTreeNode[] = [
  {
    id: 'project',
    label: 'Projekt',
    data: { owner: 'Team UI' },
    children: [
      { id: 'buttons', label: 'Buttons', data: { owner: 'Lea', status: 'Fertig' } },
      { id: 'archive', label: 'Archiv', disabled: true },
    ],
  },
];

const columns: UiTreeColumn[] = [
  { key: 'owner', label: 'Verantwortlich' },
  { key: 'status', label: 'Status' },
];
```

Knotenfeld	Bedeutung
`id`	Stabile, im gesamten Baum eindeutige String-ID
`label`	Sichtbarer Name
`children?`	Kindknoten; ohne Kinder ist der Knoten ein Blatt
`disabled?`	Sperrt den Knoten und seine Nachfahren
`data?`	Zusätzliche Tabellenwerte: String, Zahl oder null je Schlüssel

Daten als unveränderliche Arrays behandeln und bei Änderungen ein neues Array übergeben. Es werden die übergebenen Kinder angezeigt; Nachladen ist Aufgabe der konsumierenden Anwendung. IDs bleiben bei Datenaktualisierungen stabil. Die Anwendung hält die Zustands-IDs passend zu ihrer aktuellen Datenquelle.

Tree einbinden

```
import { Component } from '@angular/core';
import { Tree, type UiTreeNode } from '@creaera/ui';
```

```
@Component({
  selector: 'app-project-tree',
  imports: [Tree],
  template: `
    <ui-tree
      label="Projektstruktur"
      [nodes]="nodes"
      [(expanded)]="openIds"
      [(selected)]="selectedId"
    />
    <p>Auswahl: {{ selectedId || 'Keine' }}</p>
  `,
})
export class ProjectTree {
  nodes: UiTreeNode[] = [
    { id: 'project', label: 'Projekt', children: [{ id: 'buttons', label: 'Buttons' }] },
  ];
  openIds: readonly string[] = ['project'];
  selectedId: string | null = null;
}
```

`label` ist für die zugängliche Beschriftung erforderlich. Ohne initiale `expanded`-Werte sind alle Knoten geschlossen. `selected` startet mit null. Ein Klick auf einen Namen wählt genau diesen Knoten aus. Ein Klick auf den Pfeil öffnet oder schließt ihn, ohne die Auswahl zu verändern.

Tree Table einbinden

`TreeTable` in die `imports` der eigenen Angular-Komponente aufnehmen:

```
<ui-tree-table
  label="Projekte und Zuständigkeiten"
  [nodes]="nodes"
  [columns]="columns"
  [(expanded)]="openIds"
  [(selected)]="selectedId"
/>
```

Die erste Spalte **Name** zeigt immer `node.label` mit Einrückung und Pfeil. `columns` beschreibt die zusätzlichen Spalten in ihrer Darstellungsreihenfolge. Jede Spalte hat einen eindeutigen `key` und eine sichtbare `label`. Werte werden aus `node.data[key]` gelesen. Fehlende Werte oder null erscheinen als „—“; die Zahl 0 bleibt 0. Texte werden als Text dargestellt, nicht als HTML ausgeführt.

Die Tabelle verwendet Zeilenfokus: Pfeiltasten steuern die Hierarchie und die Zeilen. Einzelne Datenzellen sind rein lesend. Bei wenig Platz kann der Tabellencontainer horizontal scrollen. Es gibt keinen lokalen Größenparameter; das globale Größenprofil steuert Schrift und Zeilenabstände.

Gemeinsame API

Input / Binding	Bedeutung
`nodes`	Erforderliches Array von `UiTreeNode`

Input / Binding	Bedeutung
`label`	Erforderlicher zugänglicher Name des Controls
`[(expanded)]`	Array offener Knoten-IDs; Standard `[]`
`[(selected)]`	Einzelne ausgewählte Knoten-ID oder null; Standard null
`disabled`	Gesamtes Control sperren; Standard false
`columns`	Nur Tree Table: erforderliches Array zusätzlicher Spalten

Angulars Model-Bindings stellen auch die Events `(expandedChange)` und `(selectedChange)` bereit, wenn einseitige Bindung mit eigenem Eventhandler gewünscht ist. Tree und Tree Table können sich dieselben Zustandsfelder teilen oder unabhängig voneinander betrieben werden. Kein `ngModel` oder `formControl` erforderlich.

Tastaturbedienung

Taste	Verhalten
Tab	Betritt das Control an genau einem verfügbaren Knoten; weiterer Tab verlässt es
Pfeil abwärts / aufwärts	Nächsten / vorherigen sichtbaren, verfügbaren Knoten fokussieren
Pfeil rechts	Geschlossenen Knoten öffnen; bei offenem Knoten zum ersten verfügbaren Kind wechseln
Pfeil links	Offenen Knoten schließen; sonst zum verfügbaren Elternknoten wechseln
Home / End	Ersten / letzten sichtbaren, verfügbaren Knoten fokussieren
Enter / Leertaste	Fokussierten Knoten auswählen
Einzelner Buchstabe	Nächsten sichtbaren Knoten mit diesem Anfangsbuchstaben fokussieren, mit Umlauf

Fokus und Auswahl sind getrennt. Navigation wählt nicht automatisch aus. Gesperrte Knoten und ihre Nachfahren werden bei der Navigation übersprungen. Nur Knoten mit Kindern erhalten einen Pfeil und `aria-expanded`.

Tree verwendet `role="tree"` mit `treeitem`, Tree Table `role="treegrid"` mit Zeilen und Spaltenüberschriften. Ebene, Position innerhalb der Geschwister, Auswahl und Sperrstatus sind für assistive Technologien ausgezeichnet.

Leere, gesperrte und geschlossene Zustände

Ein leeres Array zeigt „Keine Einträge vorhanden.“ an. Bei einer leeren Tree Table bleiben die Spaltenüberschriften sichtbar. `[disabled]="true"` sperrt Auswahl und Auf-/Zuklappen. Ein individuell gesperrter Elternknoten sperrt auch seine Kinder, selbst wenn diese durch externen Zustand sichtbar sind.

Beim Zuklappen bleiben Daten, Auswahl und Aufklappzustände tieferer Knoten bestehen. Erneutes Öffnen stellt diesen Zustand wieder dar. Die ausgewählte ID kann dadurch vorübergehend zu einem nicht sichtbaren Knoten gehören.

In Storybook ausprobieren

Unter **Components → Tree** und **Components → Tree Table** stehen die Varianten Default, Collapsed, Selected, Disabled und Empty bereit. Die Beispiele zeigen mehrere Ebenen und einen gesperrten Zweig. In der Toolbar Light/Dark Mode und die Größenprofile wechseln; für die Tree Table auch einen schmalen Viewport testen.

Formularfelder

Ein Feld besteht aus **FormField** und einem nativen `input` oder `textarea` mit der Direktive **Input** (`uiInput`). Das Label steht klein links über dem Feld. Die Border verwendet die Primärfarbe; im Fehlerfall verwenden Border, Fokusindikator und Fehlermeldung die zentrale Fehlerfarbe.

Einstieg

`FormField` und `Input` aus `@creaera/ui` in die `imports` der eigenen Standalone-Komponente aufnehmen:

```
import { Component } from '@angular/core';
import { FormField, Input } from '@creaera/ui';

@Component({
  selector: 'app-contact',
  imports: [FormField, Input],
  template: `
    <ui-form-field label="Name">
      <input uiInput autocomplete="name" placeholder="Vor- und Nachname" />
    </ui-form-field>
    <ui-form-field label="Nachricht">
      <textarea uiInput rows="4" placeholder="Deine Nachricht"></textarea>
    </ui-form-field>
  `,
})
export class Contact {}
```

Pro `ui-form-field` genau ein direktes `input uiInput` oder `textarea uiInput` verwenden. Das Label wird automatisch über `for` und eine eindeutige Feld-ID verbunden. Ein eigenes `id` kann als natives Attribut angegeben werden. Der Placeholder ersetzt das Label nicht.

Unterschiedliche Eingabetypen

Die Eingabe bleibt ein natives HTML-Feld. Es braucht keine zusätzliche Komponentenvariante oder eigene Konfiguration für jeden Typ:

```
<ui-form-field label="E-Mail">
  <input uiInput type="email" autocomplete="email" />
</ui-form-field>
<ui-form-field label="Passwort">
  <input uiInput type="password" autocomplete="new-password" />
</ui-form-field>
<ui-form-field label="Anzahl">
  <input uiInput type="number" min="0" step="1" />
</ui-form-field>
<ui-form-field label="Datum">
  <input uiInput type="date" />
</ui-form-field>
```

Weitere native Typen wie `tel`, `url` und `search` funktionieren genauso. `type`, `name`, `autocomplete`, `placeholder`, `required`, `min`, `max`, `step`, `minlength`, `maxlength`, `readonly`, `disabled` und native Events bleiben nutzbar. Browserdarstellungen etwa des Datum-Pickers können sich unterscheiden.

Reactive Forms

Die nativen Angular-Value-Accessor bleiben erhalten. Ein Zahlenfeld liefert mit `type="number"` Zahlen bzw. null für eine leere Eingabe; es wird kein eigener Wertadapter benötigt.

```
import { Component } from '@angular/core';
import { FormControl, FormGroup, ReactiveFormsModule, Validators } from '@angular/forms';
import { FormField, Input, Button } from '@creaera/ui';

@Component({
  selector: 'app-contact-form',
  imports: [ReactiveFormsModule, FormField, Input, Button],
  template: `
    <form [formGroup]="form" (ngSubmit)="submit()" novalidate>
      <ui-form-field label="E-Mail">
        <input uiInput type="email" formControlName="email" autocomplete="email" />
      </ui-form-field>
      <ui-form-field label="Nachricht">
        <textarea uiInput formControlName="message" rows="4"></textarea>
      </ui-form-field>
      <button uiButton type="submit">Absenden</button>
    </form>
  `,
})
export class ContactForm {
  form = new FormGroup({
    email: new FormControl('', {
      nonNullable: true,
      validators: [Validators.required, Validators.email],
    }),
    message: new FormControl('', {
      nonNullable: true,
      validators: [Validators.required],
    }),
  });

  submit() {
    this.form.markAllAsTouched();
    if (this.form.valid) {
      // Fachliche Aktion der Anwendung
    }
  }
}
```

Das Feld zeigt bei einem ungültigen Angular-Control nach der ersten Änderung (`dirty`) oder nach Verlassen des Feldes (`touched`) automatisch den Fehlerzustand. Unberührte, leere Pflichtfelder sind zunächst neutral. Ohne individuelle Meldung erscheint „Bitte Eingabe prüfen.“. `markAllAsTouched()` macht Fehler bei einem Absendeversuch sichtbar. `reset()` stellt den unberührten Zustand wieder her.

Die fachlichen Validatoren definiert die Anwendung. `type="email"` allein ersetzt bei einem Reactive FormControl keinen expliziten `Validators.email`.

Template-driven Forms

`FormsModule` importieren und die Bindung am nativen Feld setzen:

```
<ui-form-field label="Name">
  <input uiInput name="name" [(ngModel)]="name" required />
</ui-form-field>
```

`name` ist ein Feld der konsumierenden Komponente. Innerhalb eines `form` braucht jedes `ngModel` einen eindeutigen Namen. Bei Verwendung von `[formControl]` oder `formControlName` wird stattdessen `ReactiveFormsModule` benötigt.

Eigene Fehlermeldungen

Ein nicht leerer `error`-Text aktiviert den Fehlerzustand unmittelbar. Das eignet sich für eine eigene Validierung oder einen Serverfehler:

```
<ui-form-field label="E-Mail" [error]="emailError">
  <input uiInput type="email" />
</ui-form-field>
```

`emailError` ist ein String; `''` entfernt den expliziten Fehler. Bei weiterhin ungültigem und bereits bearbeitetem Angular-Control bleibt der automatische Fehlerzustand bestehen. Die Anwendung entscheidet, wann eine eigene Meldung nach einer Änderung entfernt wird.

Ohne Angular-Formularbindung wird die Fehlermeldung über `error` gesteuert. Native HTML-Validierung bleibt verfügbar, wird aber nicht in eigene fachliche Fehlermeldungen übersetzt. Im Storybook ist unter **Reactive Form** ein vollständiges Formularbeispiel vorhanden.

Disabled und Readonly

```
<ui-form-field label="Name">
  <input uiInput disabled value="Nicht verfügbar" />
</ui-form-field>
<ui-form-field label="Referenz">
  <input uiInput readonly value="UI-2026-001" />
</ui-form-field>
```

Bei Angular Forms `control.disable()` / `control.enable()` verwenden. Readonly bleibt fokussierbar und sein Wert kann kopiert werden. Disabled folgt dem nativen HTML-Verhalten. Ein deaktiviertes Angular-Control löst keinen automatischen Validierungsfehler aus.

Zugänglichkeit

- `label` ist erforderlich und bleibt stets oberhalb sichtbar.

- IDs werden automatisch erzeugt oder über `id` vorgegeben.
- Fehler setzen `aria-invalid="true"` und werden über `aria-describedby` verknüpft.
- Zusätzliches Hilfswissen kann über `aria-describedby="help-id"` ergänzt werden; die Fehlerbeschreibung wird angehängt.
- Meldungen verwenden eine höfliche Live-Region. Der Fehler wird als Text und durch die Border dargestellt.

Zentrale Gestaltung

Token	Verwendung
`--ui-interaction-primary`	Normale Border und Fokusfarbe
`--ui-status-dangerous`	Fehlerborder, Fehlerfokus und Fehlermeldung
`--ui-field-label-size`	Kleine Label- und Fehlermeldungsschrift; Standard `0.75rem`
`--ui-background-secondary`	Feldfläche
`--ui-font-default`	Feldtext und Label
`--ui-font-size`	Eingabeschrift nach Größenprofil
`--ui-button-height`	Mindesthöhe einzeiliger Felder nach Größenprofil
`--ui-padding` / `--ui-radius`	Innenabstand und Rundung

Die Tokens stehen in `libs/shared/ui/src/theme.css`. Light/Dark Mode und die beiden Größenprofile gelten auch für diese Felder. Textareas sind vertikal vergrößerbar und haben eine Mindesthöhe von 7rem; `rows` steuert zusätzlich ihre native Ausgangshöhe.

Autocomplete

`Autocomplete` ergänzt ein Textfeld um eine gefilterte Vorschlagsliste. Label, Border und Fehleranzeige folgen den Formularfeldern: kleines Label links oben, Primärfarbe für die normale Border und die zentrale Fehlerfarbe bei Fehlern.

Einfacher Einstieg

```
import { Component } from '@angular/core';
import { Autocomplete } from '@creaera/ui';

@Component({
  selector: 'app-city',
  imports: [Autocomplete],
  template: `
    <ui-autocomplete label="Stadt" [options]="cities" [(value)]="city" placeholder="Stadt suchen" />
  `,
})
export class City {
  cities = ['Berlin', 'Bern', 'Bremen', 'Hamburg', 'München'];
  city = '';
}
```

Die Vorschläge sind Strings. Der gewählte String wird als Eingabewert übernommen. Groß-/Kleinschreibung wird beim Filtern ignoriert; gesucht wird nach enthaltenem Text. Doppelte identische Vorschläge werden zusammengefasst.

Freier Text bleibt erlaubt. Die Liste bietet Vorschläge und erzwingt keine Auswahl. Wenn die Anwendung ausschließlich vorhandene Werte erlauben soll, kann sie dafür einen eigenen Angular-Validator hinterlegen.

API

Input / Binding	Bedeutung
`label`	Erforderliche sichtbare und zugängliche Feldbeschriftung
`options`	Erforderliches Array von Textvorschlägen (`readonly string[]`)
`[(value)]`	Aktueller Text; Standard `''`
`placeholder`	Optionaler Placeholder
`error`	Eigene Fehlermeldung; ein nicht leerer Text aktiviert den Fehlerzustand
`disabled`	Feld und Vorschlagsauswahl sperren

Alternativ zu `[(value)]` kann Angular Forms verwendet werden. Pro Feld eine Zustandsquelle wählen, also entweder `value`-Binding oder eine Formularbindung. Der Fokus bleibt während der Listennavigation auf dem Eingabefeld.

Angular Forms

```
import { Component } from '@angular/core';
import { FormControl, ReactiveFormsModule, Validators } from '@angular/forms';
import { Autocomplete } from '@creaera/ui';

@Component({
  selector: 'app-city-form',
  imports: [ReactiveFormsModule, Autocomplete],
  template: ` <ui-autocomplete label="Stadt" [options]="cities" [formControl]="city" /> `,
})
export class CityForm {
  cities = ['Berlin', 'Bern', 'Hamburg'];
  city = new FormControl('', { nonNullable: true, validators: [Validators.required] });
}
```

`formControlName` innerhalb einer FormGroup und `[(ngModel)]` mit `FormsModule` werden ebenfalls unterstützt. Das Control speichert einen String, kein Optionsobjekt. Eingeben und Auswählen melden den neuen Wert an Angular Forms. Verlassen des Feldes markiert es als berührt.

Ein ungültiges Control zeigt nach `dirty` oder `touched` automatisch „Bitte Eingabe prüfen.“ an. Eine eigene Meldung kann über `error` gesetzt werden. Beim Absenden bei Bedarf `form.markAllAsTouched()` aufrufen. `reset()` setzt den Wert und Formularstatus zurück; `disable()` schließt die Liste und sperrt das Feld.

Tastatur und Maus

Aktion	Verhalten
Feld fokussieren	Passende Vorschläge öffnen
Tippen	Vorschläge lokal filtern, ohne automatisch einen Wert auszuwählen
Pfeil abwärts / aufwärts	Liste öffnen und zwischen Vorschlägen wechseln, mit Umlauf
Home / End bei aktivem Vorschlag	Ersten / letzten Vorschlag aktivieren
Enter bei aktivem Vorschlag	Vorschlag übernehmen und Liste schließen
Enter ohne aktiven Vorschlag	Freien Text behalten; normales Absenden eines Formulars bleibt möglich
Escape	Liste schließen, eingegebenen Text behalten
Tab	Liste schließen und zum nächsten Control wechseln; nichts automatisch übernehmen

Aktion	Verhalten
Klick auf Vorschlag	Vorschlag übernehmen, Fokus bleibt im Textfeld
Klick außerhalb / Verlassen	Liste schließen

Wenn nichts passt, erscheint „Keine Vorschläge gefunden.“. Das Feld bleibt editierbar. Der aktive Vorschlag und die Listenzuordnung werden über `aria-activedescendant`, `aria-controls` und `aria-expanded` vermittelt. Die Vorschläge haben Listbox-/Option-Semantik. Fehler werden mit dem Feld verbunden.

Vorschläge und Darstellung anpassen

Die Anwendung übergibt ein neues `options`-Array, wenn sich ihre Daten ändern. Die Komponente filtert dieses Array lokal. Daten laden und fachliche Validierung bleiben Aufgabe der Anwendung.

Theme und Größenprofil gelten wie bei den Formularfeldern. Die Liste erscheint unter dem Feld, hat eine begrenzte Höhe und scrollt bei vielen Vorschlägen. Für Container mit `overflow: hidden` muss die Anwendung ausreichend Platz vorsehen, damit das lokale Popup nicht abgeschnitten wird.

In Storybook ausprobieren

Unter **Components → Autocomplete** gibt es Default, Prefilled, No Results, Error, Disabled und Reactive Form. Zum Beispiel `Ber` tippen, mit den Pfeiltasten einen Vorschlag aktivieren und Enter drücken. In der Toolbar beide Themes und die Größenprofile testen.

Creaera UI

Angular-Komponenten mit einer kleinen API, Tailwind CSS und Storybook in einem Nx-Workspace. Angular 22 · Nx 24 · Tailwind CSS 4 · Storybook 10.

Anwenderdokumentation in Storybook

Unter **Anwenderdoku** stehen die Anleitung zum Einstieg, die vollständige Farbkonfiguration, Light/Dark Mode, responsive Größen und die API-/Projektreferenz. Die Referenz übernimmt diese README direkt, damit beide Stellen synchron bleiben.

Entwicklung

Node.js 24 (mindestens 24.15) und npm verwenden:

```
npm ci
npm run storybook
npm run build
npm test
npm run build-storybook
```

Storybook läuft auf Port 6006. In einer Cloud-Umgebung ist für einen Zugriff von außen eine entsprechende Portfreigabe erforderlich.

- `libs/shared/ui`: eigenständige, paketierbare Komponentenbibliothek `@creaera/ui`.
- `libs/storybook`: Stories, Dokumentation und Theme-/Größen-Toolbar.
- `AGENTS.md`: Anforderungen und Arbeitsregeln.

Zentrale Konfiguration

`libs/shared/ui/src/theme.css` enthält alle Farbrollen und Größen. Hier werden Farben, Paddings, Margins, Buttongrößen und die beiden Größenprofile angepasst. Die CSS-Variablen gelten für alle Komponenten. Der Breakpoint liegt zunächst bei 48rem und steht ebenfalls in dieser Datei.

Am `body` werden `ui-light` oder `ui-dark` gesetzt. Ohne Klasse gilt Light Mode. Ohne Größenklasse wird die Bildschirmbreite verwendet. `ui-small` bzw. `ui-large` überschreiben die automatische Größenwahl. Diese Klassen können auch einen Teilbaum thematisieren.

```
<body class="ui-dark ui-large"></body>
```

`styles.css` lädt Tailwind und die zentralen Tokens. Im Workspace diese Datei in die globalen Styles der Anwendung aufnehmen. Das gebaute Paket liefert die beiden CSS-Dateien mit; für Tailwind-Verbraucher `@creaera/ui/styles.css` über deren Tailwind/PostCSS-Pipeline importieren. Für andere Verbraucher zuerst `npx @tailwindcss/cli -i libs/shared/ui/src/styles.css -o dist/ui.css` ausführen und das erzeugte CSS laden.

Kleine öffentliche API

Alle Komponenten und Direktiven sind standalone und aus `@creaera/ui` importierbar. Kein globales Angular-Modul und keine providers-Konfiguration erforderlich. Native HTML-Attribute und Events werden verwendet, wo sie bereits ausreichen.

Export	Verwendung	Minimale API
`Button`	`<button uiButton>`	`variant`: primary (Default), secondary, tertiary, text; `type`: button (Default), submit, reset; native disabled/click
`Checkbox`	`<ui-checkbox>Beschriftung</ui-checkbox>`	`[(checked)]`, disabled; alternativ ngModel oder FormControl
`ToggleSwitch`	`<ui-switch>Beschriftung</ui-switch>`	`[(checked)]`, disabled; alternativ ngModel oder FormControl
`Radio`	`<input type="radio" uiRadio>`	native name/value/disabled; ngModel oder FormControl
`ButtonGroup`	`<ui-button-group>`	options, label, `[(selected)]` (string[]), multiple (Default false), disabled
`Chip`	`<ui-chip>`	disabled; `(removed)` beim Klick auf das X; Elternkomponente entfernt den Chip
`Card`	`<ui-card>`	collapsible (Default false), `[(expanded)]` (Default false), border (primary/secondary), label für die Toggle-Beschriftung
`Dropdown`	`<ui-dropdown>`	label, options, disabled; `(selected)` liefert den value
`OptionButton`	`<ui-option-button>`	wie Dropdown, zusätzlich `(action)` für den Hauptbutton

`UiOption` ist `{ value: string; label: string; disabled?: boolean }`. Werte müssen innerhalb einer Optionsliste eindeutig sein. In Button Groups ist `selected` auch bei Einfachauswahl ein Array (höchstens ein Eintrag).

```
import { Component } from '@angular/core';
import { Button, Checkbox, Dropdown } from '@creaera/ui';

@Component({
```

```
  selector: 'app-example',
  imports: [Button, Checkbox, Dropdown],
  template: `
    <button uiButton variant="secondary" (click)="save()">Speichern</button>
    <ui-checkbox [(checked)]="enabled">Benachrichtigungen</ui-checkbox>
    <ui-dropdown label="Aktionen" [options]="options" (selected)="handle($event)" />
  `,
})
export class Example {
  enabled = false;
  options = [{ value: 'copy', label: 'Duplizieren' }];
  save() {
    /* Anwendungsaktion */
  }
  handle(value: string) {
    /* Menüaktion */
  }
}
```

Icons werden als SVG oder anderes Element mit `uiIcon` projiziert. Icon links, Text rechts ergibt sich aus der Reihenfolge. Icons bleiben weiß. Icon-only Buttons benötigen ein `aria-label`; dekorative Icons `aria-hidden="true"`.

```
<button uiButton aria-label="Hinzufügen">
  <svg uiIcon aria-hidden="true" viewBox="0 0 24 24">
    <path d="M12 5v14M5 12h14" stroke="currentColor" stroke-width="2" />
  </svg>
</button>
```

Bedienung und Annahmen

- Buttons und Chips: Hover, Pressed und Focused sind echte CSS-Zustände. Mit Maus, Leertaste und Tab in Storybook ausprobieren.
- Checkbox/Switch: integriertes Label, native Checkbox-Semantik, Forms-Unterstützung.
- Radio: `type="radio"` muss im Template stehen, damit Angular den korrekten Forms-Accessor auswählt. Ein gemeinsames `name` gruppiert Radios.
- Button Group: Einfachauswahl mit Radiogroup-Semantik, Pfeiltasten, Home/End; Mehrfachauswahl mit Toggle-Buttons und `aria-pressed`.
- Menüs sind **Aktionsmenüs**, keine Select-/Formularfelder: Enter/Leertaste oder Pfeiltasten öffnen; Pfeile und Home/End navigieren; Escape schließt und setzt den Fokus zurück. Tab und ein Klick außerhalb schließen ebenfalls.
- „Active“ bei Checkbox/Radio ist vorerst der bedienbare, nicht ausgewählte Zustand.
- Textbuttons bleiben auch bei Hover/Pressed transparent. Die weiße Border und der Fokusindikator folgen den anderen Buttons. Der Text folgt der Theme-Farbe.
- Es gibt bewusst keine Inputs für lokale Farben, Größen oder zusätzliche Chip-Varianten. Formularfelder sind ebenfalls verfügbar.

Tests

`npm test` führt Angular-kompilierte Vitest-Tests mit jsdom aus. Sie prüfen native Button-Semantik, Forms, Auswahlzustände und Menü-/Tastaturinteraktionen.

`npm run test:e2e` baut Storybook und prüft die echten Browseransichten sowie Themes/Größenprofile mit Playwright. Einmalig `npx playwright install chromium` ausführen oder `PLAYWRIGHT_CHROMIUM_EXECUTABLE_PATH` auf einen vorhandenen Chromium setzen. In CI wird Chromium mit Systemabhängigkeiten installiert.

Entfernbare Chips und Cards

Chips verwenden jetzt einen Container mit einem separaten X-Button statt `<button uiChip>`. So entstehen keine verschachtelten Buttons:

```
@if (showTag) {
<ui-chip (removed)="showTag = false">Designsystem</ui-chip>
}
```

`removed` meldet den Entfernungswunsch; die konsumierende Komponente aktualisiert ihre Liste. `[disabled]="true"` sperrt das X. Der zugängliche Name enthält den sichtbaren Chip-Text.

Cards sind weiß mit primärer oder optional sekundärer (grauer) Border, auch im Dark Mode. `uiCardHeader` markiert den immer sichtbaren Kopf. Der übrige Inhalt wird bei ausklappbaren Cards ein-/ausgeblendet. Der Pfeil liegt unten mittig; die ausklappbare Variante sieht durch zwei zusätzliche Ebenen wie drei gestapelte Cards aus. Cards können ohne besondere Konfiguration ineinander verschachtelt werden, inklusive unabhängiger ausklappbarer Cards.

```
<ui-card>
  <h2 uiCardHeader>Übersicht</h2>
  <ui-card collapsible border="secondary" label="Projektdetails" [(expanded)]="detailsOpen">
    <h3 uiCardHeader>Projektdetails</h3>
    <p>Inhalt der inneren Card.</p>
  </ui-card>
</ui-card>
```

`Card` aus `@creaera/ui` importieren. Ohne `collapsible` ist der Inhalt immer sichtbar. `expanded` startet mit false, steuert bei ausklappbaren Cards den Inhalt und unterstützt Two-Way-Binding. Ein-/Ausklappen behält Kindzustände bei. `label` sollte die Card benennen; der Button ergänzt „ausklappen“ / „einklappen“. `uiCardHeader` ist ein Projektionsattribut ohne zusätzlichen Angular-Import. `--ui-card-background` und `--ui-card-text` in `theme.css` steuern die weiße Card-Fläche und deren dunklen Text zentral.

Formularfelder

`FormField` und `Input` sind Standalone-Exporte aus `@creaera/ui`. Ein kleines Label steht links über dem Feld; Border und Fokus verwenden die Primärfarbe, im Fehlerfall `--ui-status-dangerous`.

```
<ui-form-field label="E-Mail" [error]="emailError">
  <input uiInput type="email" autocomplete="email" [formControl]="email" />
</ui-form-field>
<ui-form-field label="Nachricht">
  <textarea uiInput rows="4" [(ngModel)]="message"></textarea>
</ui-form-field>
```

`label` ist erforderlich; `error` ist ein optionaler Meldungstext (Standard `''`). Pro FormField ein direktes natives Feld verwenden. Die Bindung erfolgt am `input`/`textarea`; dadurch bleiben HTML-Attribute, Events sowie Angular Forms inklusive Zahlenwerten erhalten. Für Reactive Forms `ReactiveFormsModule`, für `ngModel` `FormsModule` importieren.

Ungültige Angular-Controls erhalten nach `dirty` oder `touched` automatisch eine rote Border und eine allgemeine Fehlermeldung. Eigene Texte über `error` setzen; ein nicht leerer Text aktiviert den Fehlerzustand sofort. Die Anwendung definiert die Validatoren und ruft bei Bedarf `form.markAllAsTouched()` beim Absenden auf. Ohne Angular Forms steuert `error` den Fehlerzustand. Label, Feld-ID und Fehlerbeschreibung sind automatisch zugänglich verknüpft.

Storybook enthält Text-, E-Mail-, Passwort-, Zahlen- und Datumsfelder, Textareas, Fehler-, Disabled- und Readonly-Zustände sowie ein vollständiges Formularbeispiel. Die Anleitung steht unter **Anwenderdoku → Formularfelder**. Labelgröße und Farben werden zentral über `theme.css` konfiguriert.

Autocomplete

`Autocomplete` ergänzt die Formularfelder um Textvorschläge mit lokaler Filterung.

```
<ui-autocomplete label="Stadt" [options]="cities" [(value)]="city" placeholder="Stadt suchen" />
```

`Autocomplete` aus `@creaera/ui` importieren. `cities` ist ein `readonly string[]`, `city` ein String. `label` und `options` sind erforderlich; `placeholder`, `error` und `disabled` optional. Angular Forms (`formControl`, `formControlName`, `ngModel`) kann alternativ zum Value-Binding verwendet werden. Freier Text ist erlaubt; fachliche Validatoren definiert die Anwendung. Pfeiltasten, Home/End, Enter, Escape und Tab bedienen die Vorschlagsliste; der Fokus bleibt im Eingabefeld. Die vollständige Anleitung steht unter **Anwenderdoku → Autocomplete**.

Tree und Tree Table

`Tree` (`<ui-tree>`) und `TreeTable` (`<ui-tree-table>`) verwenden dieselbe kleine API: `nodes`, `label`, `[(expanded)]` als ID-Array, `[(selected)]` als einzelne ID oder null und `disabled`. Die Tree Table erhält zusätzlich `columns`; ihre erste Spalte „Name“ zeigt immer die Hierarchie. Zusätzliche Spalten lesen Werte aus `node.data[column.key]` und zeigen bei fehlenden Werten „—“ an.

```
import type { UiTreeNode, UiTreeColumn } from '@creaera/ui';

const nodes: UiTreeNode[] = [
  {
```

```
    id: 'project',
    label: 'Projekt',
    children: [{ id: 'buttons', label: 'Buttons', data: { status: 'Fertig' } }],
  },
];
const columns: UiTreeColumn[] = [{ key: 'status', label: 'Status' }];
```

```
<ui-tree label="Projektstruktur" [nodes]="nodes" [(expanded)]="openIds" [(selected)]="selectedId" />
<ui-tree-table
  label="Projektübersicht"
  [nodes]="nodes"
  [columns]="columns"
  [(expanded)]="openIds"
  [(selected)]="selectedId"
/>
```

Die Daten und Zustandsfelder gehören in die konsumierende Angular-Komponente; `Tree` bzw. `TreeTable` in deren `imports` aufnehmen. Initial z. B. `openIds: readonly string[] = ['project']` und `selectedId: string | null = null`. IDs müssen im gesamten Baum eindeutig sein; Spaltenschlüssel innerhalb einer Tree Table ebenfalls. Kinder werden über `children` angegeben. `disabled` am Knoten sperrt ihn und seine Nachfahren. Die Pfeiltasten steuern den Fokus sowie das Auf-/Zuklappen; Enter oder Leertaste wählen aus. Die Auswahl folgt nicht automatisch dem Fokus. Ein Klick auf einen Pfeil ändert nur die Aufklappzustände. Home/End und Navigation über Anfangsbuchstaben sind unterstützt. Ausgeblendete Knoten bleiben in der Datenquelle und behalten ihren Auswahl-/Aufklappzustand.

Tree hat ARIA-Tree-Semantik, Tree Table eine zeilenfokussierte Treegrid-Semantik. Zellen sind rein lesend; kein Forms-Adapter ist nötig. Weitere Beispiele und die genaue Tastaturbelegung stehen in Storybook unter **Anwenderdoku → Trees**.

Architekturleitfaden

Storybook enthält unter **Architektur** neun Kapitel zu modularen Angular-/Nx- Anwendungen, Komponentengrenzen, State-Besitz, NgRx Signal Store, CRUD- und Aktions-APIs, Server-Suche und Tests. Ein zusammenhängendes Projektverwaltungs- Beispiel veranschaulicht die Entscheidungen.

Die kompilierten Referenzdateien liegen unter `libs/storybook/examples/architecture`. Die vorgeschlagenen Fach-Libraries und Backend-Endpunkte sind ein Zielbild für konsumierende Anwendungen; dieses Repository enthält weiterhin die UI-Bibliothek und Storybook.

```
npm run docs:architecture:sync
npm run docs:architecture:check
npm run test:architecture
```

Codeblöcke in den Kapiteln werden aus den echten Beispieldateien übernommen. Die separaten Tests verwenden Angulars HTTP-Testbackend und prüfen auch Stornierung, Nebenläufigkeit und Injector-Lifecycle. NgRx 22.0.1 und der Router sind Entwicklungsabhängigkeiten für diese Beispiele. Das veröffentlichbare UI-Paket erhält keine NgRx-Laufzeitabhängigkeit.

Moderne Angular-Architektur

Dieser Leitfaden zeigt eine pragmatische modulare Architektur mit **Angular 22, Nx 24 und NgRx Signal Store 22**. Durchgehend geht es um eine Projektverwaltung: Projekte suchen, anlegen, ändern, löschen und veröffentlichen.

Was hier tatsächlich vorhanden ist

Die produktive Bibliothek bleibt unter `libs/shared/ui`. Die Architektur ist eine Anleitung für konsumierende Anwendungen, kein bereits eingerichtetes Backend und keine neue fachliche Anwendung in diesem Repository.

Die vollständigen, kompilierten Referenzdateien liegen unter `libs/storybook/examples/architecture`. Ihre HTTP- und Store-Tests verwenden Angulars HTTP-Testbackend. Die Beispiele wurden mit Angular **22.2.3**, NgRx **22.0.1** und den im Lockfile festgelegten Werkzeugen geprüft. Stand: **24. Oktober 2026**. Architekturentscheidungen hängen zusätzlich von Teamgröße, Datenmengen und fachlichen Grenzen ab.

Ein durchgehender Datenfluss

```
Benutzeraktion
  → Feature-Page / Container
  → fachliche Methode am Signal Store
  → typisierter API-Client
  → HTTP / Server mit Geschäftsregeln
  → DTO-Mapping
  → Store-Zustand
  → computed View-State
  → UI-Komponenten
```

Eine UI-Komponente meldet zum Beispiel `publishRequested(project)`. Der Container ruft `store.publish(project, requestId)` auf. Der API-Client sendet den Befehl. Nach bestätigtem Erfolg lädt der Store die aktuelle Liste neu. Der Server prüft, ob das Projekt veröffentlicht werden darf.

Die neun Kapitel

Kapitel	Entscheidung, die es erklärt
Start	Gesamtbild, Umfang und Quellen
Komponenten anlegen und CLI	Generatoren, Library-Export, Stories und wichtige Befehle
Module und Nx	Fachliche Grenzen, Libraries, Imports und Lazy Loading

Kapitel	Entscheidung, die es erklärt
Komponenten und APIs	Container, fachliche UI, wiederverwendbare Controls und Formulare
State und Signal Store	Zustandsbesitz, Lebensdauer, Entities, Methoden und abgeleitete Werte
HTTP und CRUD	Ressourcen, DTOs, Clients, Pagination und Konfliktbehandlung
Aktionen und Nebenläufigkeit	Fachliche Commands, Idempotenz und RxJS-Operatoren
Server-Suche	Debounce, sofortige Stornierung alter Reads und Autocomplete-Anbindung
Tests und Weiterentwicklung	Ausführbare Beispiele, sinnvolle Tests und schrittweiser Ausbau

Ein sinnvoller Startumfang

Mit einer Fachfunktion, einem Feature-Store, einem API-Client und wenigen fachlichen UI-Komponenten beginnen. Libraries erst an stabilen Grenzen herauslösen. Ein Store pro Button oder eine Library pro kleiner Datei bringt meist keine zusätzliche Klarheit.

Die UI-Bibliothek enthält Gestaltung und generische Interaktion. Fachliche Regeln, Serverzugriff und konkrete Anwendungs-Stores liegen in der konsumierenden Anwendung. NgRx ist deshalb nur eine Entwicklungsabhängigkeit für die geprüften Dokumentationsbeispiele und keine Laufzeitabhängigkeit von `@creaera/ui`.

Offizielle Grundlagen

Die Beispiele verwenden die aktuellen, installierten NgRx-APIs. Weiterführend:

- Angular Style Guide
- Angular Dependency Injection
- Angular HttpClient
- NgRx Signal Store
- NgRx RxJS-Integration
- NgRx Entity Management
- Nx Module Boundaries
- HTTP-Semantik, RFC 9110
- Problem Details, RFC 9457

Die NgRx-APIs wurden zusätzlich gegen die offiziellen Quellen im NgRx-Repository und die installierte Version 22.0.1 geprüft. Neuere optionale APIs sollten passend zur konkreten Version bewertet werden; der Leitfaden setzt sie nicht pauschal voraus.

Module und Nx

„Modular“ bedeutet zunächst klare Verantwortlichkeiten und gerichtete Abhängigkeiten. Moderne Angular-Anwendungen können dafür Standalone-Komponenten, Lazy Routes und Dependency Injection verwenden. Für fachliche Grenzen ist kein NgModule pro Bereich nötig.

Nach Fachlichkeit schneiden

Ein mögliches Zielbild für eine konsumierende Anwendung:

```
apps/
  admin/                        # Bootstrap, globale Provider, Root-Routen
libs/
  projects/
    feature/                    # Seiten, Routing, Ablaufsteuerung
    data-access/                # ProjectsStore, ProjectsApi, DTO-Mapping
    ui/                         # ProjectList, ProjectEditor
    domain/                     # Project, Commands, reine Fachfunktionen
  shared/
    ui/                         # bestehende generische Komponentenbibliothek
    util/                       # wenige fachneutrale Hilfen
  storybook/                    # bestehende Dokumentation und Stories
```

`apps/admin` und `libs/projects/*` sind hier ein vorgeschlagener Aufbau. Die kompilierten Beispiele liegen zur besseren Lesbarkeit gemeinsam unter `libs/storybook/examples/architecture` und verwenden dort relative Imports.

Zuerst die Fachlichkeit verstehen: Eine Rechnungsverwaltung muss nicht denselben Store oder dieselben Modelle wie eine Projektverwaltung besitzen. Gemeinsame Daten über klar benannte Verträge austauschen. Ein großer Ordner `shared` sollte nicht zur Ablage für alles werden, das zwei Dateien brauchen.

Erlaubte Abhängigkeiten

Typ	Typische erlaubte Ziele
App	Features und explizite Bootstrap-Konfiguration, Shared UI/Util
Feature	Data Access, fachliche UI, Domain, Shared UI/Util
Data Access	Domain, fachneutrale Utilities, Angular HTTP, NgRx
Fachliche UI	Domain-Typen, Shared UI/Util
Shared UI	Generische Darstellung und Interaktion; keine fachlichen Stores
Domain	Reine Typen und Funktionen; möglichst ohne Angular/HTTP

Eine fachliche UI darf `Project` als Typ kennen. Ein generischer Button kennt keinen `ProjectsStore`. Eine UI-Library importiert nicht zurück in ein Feature. Zyklische Imports sind ein Signal, die Zuständigkeiten erneut zu prüfen.

Öffentliche Einstiegspunkte

Jede Library veröffentlicht eine kleine API über ihren Einstiegspunkt. Consumer importieren beispielsweise `@creaera/projects/domain` und greifen nicht direkt auf interne Dateipfade einer anderen Library zu. Solche Aliase werden erst beim Anlegen der entsprechenden Libraries konfiguriert; sie existieren in diesem Repository noch nicht.

Anfangs reichen Exporte für die tatsächlich benötigten Typen, Komponenten, Routes und Services. Interne HTTP-DTOs müssen nicht automatisch Teil jeder öffentlichen Domain-API werden.

Grenzen durch Nx prüfen

Libraries können etwa diese Tags bekommen:

```
{
  "name": "projects-feature",
  "tags": ["scope:projects", "type:feature"]
}
```

Mit `@nx/enforce-module-boundaries` lassen sich die Regeln in ESLint durchsetzen. Beispiel für den Regelteil einer Nx-Flat-Config:

```
const boundaryRules = {
  '@nx/enforce-module-boundaries': [
    'error',
    {
      enforceBuildableLibDependency: true,
      allow: [],
      depConstraints: [
        {
          sourceTag: 'scope:projects',
          onlyDependOnLibsWithTags: ['scope:projects', 'scope:shared'],
        },
        { sourceTag: 'scope:shared', onlyDependOnLibsWithTags: ['scope:shared'] },
        {
          sourceTag: 'type:feature',
          onlyDependOnLibsWithTags: ['type:data-access', 'type:ui', 'type:domain', 'type:util'],
        },
        { sourceTag: 'type:data-access', onlyDependOnLibsWithTags: ['type:domain', 'type:util'] },
        { sourceTag: 'type:ui', onlyDependOnLibsWithTags: ['type:ui', 'type:domain', 'type:util'] },
        { sourceTag: 'type:domain', onlyDependOnLibsWithTags: ['type:domain', 'type:util'] },
        { sourceTag: 'type:util', onlyDependOnLibsWithTags: ['type:util'] },
      ],
    },
  ],
};
```

Dieses Rules-Objekt wird als `rules: boundaryRules` in die vorhandene ESLint-Konfiguration integriert, nachdem die Nx-ESLint-Unterstützung eingerichtet ist. Er ist keine bereits aktivierte Regel dieses Workspaces. Alle Libraries passend taggen und Lint in CI ausführen. Ein ungetaggtes Projekt erhält durch die Beispiele nicht versehentlich eine sinnvolle fachliche Grenze.

Lazy Loading und Bootstrap

Die Root-App stellt Infrastruktur bereit und lädt den Bereich bei Bedarf:

```
import { provideHttpClient } from '@angular/common/http';
import { type ApplicationConfig } from '@angular/core';
import { provideRouter, type Routes } from '@angular/router';

export const APP_ROUTES: Routes = [
  {
    path: 'projects',
    loadChildren: () => import('./project.routes').then((module) => module.PROJECT_ROUTES),
  },
];
export const appConfig: ApplicationConfig = {
  providers: [provideHttpClient(), provideRouter(APP_ROUTES)],
};
```

Der Bereich veröffentlicht seine eigenen Routes:

```
import { type Routes } from '@angular/router';

export const PROJECT_ROUTES: Routes = [
  { path: '', loadComponent: () => import('./project-page').then((module) => module.ProjectPage) },
];
```

Die Page stellt im Beispiel ihren Store selbst bereit. Das schafft eine klare Lebensdauer pro Seiteninstanz. Für mehrere Kindseiten kann eine gemeinsame Feature-Shell den Store bereitstellen. Bei Route-Providern, Route-Caching und einer eigenen RouteReuseStrategy die tatsächliche Lebensdauer bewusst prüfen.

Wann eine neue Library sinnvoll ist

Eine neue Library hilft bei einer stabilen fachlichen Grenze, Wiederverwendung, eigener Zuständigkeit, einem separaten Build oder einer gewünschten Abhängigkeitsregel. Für einen kleinen Bereich dürfen Feature, Store und API-Client zunächst in einer zusammenhängenden Library liegen und später gezielt getrennt werden.

Komponenten anlegen und wichtige CLI-Befehle

Diese Seite zeigt den Weg vom Generator bis zur verwendbaren und dokumentierten Komponente. Die Beispiele beziehen sich auf **Angular 22.2.3 und Nx 24.0.1**.

Welches Werkzeug verwenden?

In diesem Repository werden Projekte durch **Nx** konfiguriert. Deshalb aus dem Repository-Root `npx nx ...` verwenden. Eine klassische Angular-CLI-Anwendung mit `angular.json` verwendet dagegen `ng ...`, bei lokal installierter CLI auch `npx ng ...`.

`g` ist die Kurzform von `generate`. Generatoren erstellen oder ändern Dateien; Targets wie `build` und `test` führen konfigurierte Aufgaben aus. `--dry-run` zeigt die geplanten Änderungen, ohne sie zu schreiben. `--help` erklärt die Optionen der tatsächlich installierten Version.

Schritt 1: Ziel und kleine API festlegen

Eine generische UI-Komponente gehört hier unter `libs/shared/ui/src/lib`. Fachliche Komponenten gehören in den jeweiligen Bereich, beispielsweise in eine später angelegte `projects/ui`-Library. Vor dem Generieren klären:

- Welcher sichtbare Zweck und welche Interaktion gehören zur Komponente?
- Welche Inputs braucht ein konkreter Consumer tatsächlich?
- Gibt es eine Benutzerabsicht, die als typisierter Output gemeldet werden muss?
- Welche Darstellung ergibt sich bereits aus den zentralen Design-Tokens?

Das Beispiel ist eine reine Statusanzeige mit `label` und einem semantischen `tone`. Sie braucht kein Klick-Event, keinen Store und keinen HTTP-Client.

Schritt 2: Komponente mit Nx erzeugen

Zuerst den Dry-Run ansehen:

```
npx nx g @nx/angular:component libs/shared/ui/src/lib/status-badge/status-badge \
  --selector=ui-status-badge \
  --standalone \
  --changeDetection=OnPush \
  --style=none \
  --inlineTemplate \
  --export \
  --dry-run
```

Für die echte Anlage denselben Befehl ohne `--dry-run` ausführen. Er erzeugt hier `status-badge.ts` und `status-badge.spec.ts` im angegebenen Ordner und aktualisiert den Library-Einstiegspunkt. Er kann zusätzlich Generator-Defaults in `nx.json` anpassen; deshalb die gesamte Änderung ansehen.

Nx 24 erwartet bei diesem Generator einen **Dateipfad**, nicht das früher oft gezeigte `--project`-/`--directory`-Schema. Der Pfad bestimmt das Projekt. Andere Generatoren können weiterhin `--project` und `--path` benötigen.

Option	Warum sie im Beispiel vorkommt
`--selector=ui-status-badge`	HTML-Name mit dem UI-Präfix
`--standalone`	Nutzung über `imports`, ohne eigenes NgModule
`--changeDetection=OnPush`	Explizite Darstellungsstrategie
`--style=none`	Keine zusätzliche Stylesheet-Datei für diese kleine Anzeige
`--inlineTemplate`	Kurzes Template direkt in der TypeScript-Datei
`--export`	Standalone-Komponente aus dem Library-Einstiegspunkt exportieren

Standalone und OnPush sind bei den hier installierten Generatoren bereits Standard. Die Flags machen die Absicht sichtbar. Aktuelle Dateinamen müssen nicht auf `.component.ts` enden; das Beispiel folgt der kurzen Benennung dieses Repositories. Falls ein Team einen Suffix vereinbart, dessen Generatoroptionen explizit wählen und die Ausgabe im Dry-Run prüfen.

Schritt 3: Das Gerüst mit Verhalten füllen

Ein mögliches fertiges Beispiel:

```
import { ChangeDetectionStrategy, Component, input } from '@angular/core';

@Component({
  selector: 'ui-status-badge',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: '<span>{{ label() }}</span>',
  host: { '[style.color]': "'var(--ui-status-' + tone() + ')'" },
})
export class StatusBadge {
  readonly label = input.required<string>();
  readonly tone = input<'normal' | 'caution' | 'dangerous'>('normal');
}
```

Die Statusfarben stammen aus `theme.css`. Der Text erklärt den Zustand zusätzlich zur Farbe. `label` ist erforderlich, `tone` hat einen sinnvollen Standard. Die Komponente verändert keine fachlichen Daten.

Die Datei oben liegt hier als kompilierte **Referenz** unter `libs/storybook/examples/architecture`. Es wurde für diese Anleitung keine neue öffentliche Badge-Komponente in `libs/shared/ui` angelegt. Beim Nachbauen ersetzt man damit den Inhalt der wirklich generierten Komponentendatei.

Schritt 4: Exportieren und verwenden

Der `--export`-Schritt ergänzt den Einstiegspunkt `libs/shared/ui/src/index.ts`. Falls der Export manuell nötig ist, lautet er für diesen Beispielpfad:

```
export { StatusBadge } from './lib/status-badge/status-badge';
```

In der konsumierenden Standalone-Komponente `StatusBadge` aus `@creaera/ui` importieren und in deren `imports` aufnehmen. Danach im Template verwenden:

```
<ui-status-badge label="Bereit" />
<ui-status-badge label="Bitte prüfen" tone="caution" />
<ui-status-badge label="Fehler" tone="dangerous" />
```

Ein TypeScript-Import allein reicht für Angular-Templates nicht; die Komponente muss auch in `imports` verfügbar sein. Ein öffentliches Library-API wird über den Einstiegspunkt genutzt, nicht über den internen Dateipfad einer anderen Library.

Schritt 5: Eine Story ergänzen

Im tatsächlichen Storybook eine Datei `libs/storybook/status-badge.stories.ts` anlegen. Ein Gerüst:

```
import type { Meta, StoryObj } from '@storybook/angular';
import { StatusBadge } from './status-badge';

const meta: Meta<StatusBadge> = {
  title: 'Components/Status Badge',
  component: StatusBadge,
  tags: ['autodocs'],
  args: { label: 'Bereit', tone: 'normal' },
};
export default meta;
type Story = StoryObj<StatusBadge>;
export const Normal: Story = {};
export const Caution: Story = { args: { label: 'Bitte prüfen', tone: 'caution' } };
export const Dangerous: Story = { args: { label: 'Fehler', tone: 'dangerous' } };
```

Die kompilierte Referenz importiert ihr Badge aus der benachbarten Beispieldatei. In der echten Story nach dem Library-Export stattdessen `import { StatusBadge } from '@creaera/ui'` verwenden. Dieses Storybook findet `*.stories.ts` direkt unter `libs/storybook`; eine Story an einem anderen Ort braucht ein passendes Glob in `.storybook/main.ts`.

Die Referenz-Story ist kein zusätzlicher Eintrag im produktiven Komponentenkatalog. Beim Umsetzen echte Varianten, relevante Zustände und Interaktionen zeigen.

Schritt 6: Tests und Build prüfen

```
npx nx test ui
npx nx build ui
npm run storybook
npm run build-storybook
```

Der Generator legt eine Spec-Datei an. Diese mit Tests für das tatsächliche Verhalten füllen, etwa erforderliche Beschriftung, Interaktion, Disabled und Formularbindung. Ein bloßes `should create` ist bei komplexeren Komponenten kein ausreichender Funktionstest. Für eine reine Anzeige reichen passende, kleine Prüfungen; keine Infrastruktur nur für einen trivialen Wert aufbauen.

Häufige Generatoren in diesem Nx-Workspace

Diese Befehle sind bewusst **Dry-Runs**. Die Namen sind Beispiele für neue Dateien; sie sind noch keine zusätzlichen Library-Exporte.

```
# Fachneutrales Service-Gerüst; --injectable passt zu den @Injectable-Beispielen
npx nx g @nx/angular:service format-label --project=ui --path=libs/shared/ui/src/lib --injectable --
dry-run

# Attributdirektive: der Nx-Parameter bekommt den Namen ohne eckige Klammern
npx nx g @nx/angular:directive libs/shared/ui/src/lib/auto-focus --selector=uiAutoFocus --standalone
--dry-run

# Pipe
npx nx g @nx/angular:pipe libs/shared/ui/src/lib/display-label --standalone --dry-run
```

Der Direktiven-Generator erzeugt aus `--selector=uiAutoFocus` den Selektor `[uiAutoFocus]`. Die Beispiel-Pipe heißt bei dieser Generatorversion in der Dateiausgabe `display-label-pipe.ts`. Die tatsächlich erzeugten Namen prüfen, bevor Imports oder Exporte ergänzt werden.

API-Clients gehören in Data Access, Auth-Guards und Interceptors in die entsprechende App-/Feature-Infrastruktur. Die UI-Library ist dafür kein fachlicher Ablageort. Bei solchen Generatoren erst das richtige vorhandene Projekt bestimmen.

Neue Nx-Library oder App anlegen

Nur wenn ein neues Projekt gebraucht wird; für eine einzelne neue Komponente ist keine zusätzliche Library nötig. Die folgenden Pfade sind vorgeschlagene neue Projekte aus dem Architekturleitfaden:

```
npx nx g @nx/angular:library libs/projects/ui --name=projects-ui \
  --buildable --standalone --unitTestRunner=vitest-angular --linter=none --dry-run

npx nx g @nx/angular:application apps/admin --name=admin \
  --standalone --routing --bundler=esbuild --unitTestRunner=vitest-angular \
  --e2eTestRunner=none --linter=none --dry-run
```

Die Beispiele wählen explizit den nativen Angular-Vitest-Runner. Eine Library braucht dafür in dieser Nx-Version `--buildable` oder `--publishable`. `--linter=none` und `--e2eTestRunner=none` vermeiden für diese Vorschau zusätzliche Tool-Einrichtungen. Für eine echte Anwendung passende Lint- und E2E-Targets nach dem Teamstandard einrichten.

Die wichtigsten ng-Befehle im klassischen Angular-CLI-Projekt

Die folgenden Generatoren wurden in einem separaten kleinen CLI-Prüfworkspace mit dem Projektnamen **demo** als Dry-Run geprüft. Sie sind für einen klassischen Angular-Workspace gedacht, nicht zum direkten Generieren in diesem Nx-Repository. Bei einem eigenen Projekt `demo` durch dessen tatsächlichen Projektnamen ersetzen.

Zweck	Befehl
Komponente (`g c`)	`npx ng g c features/projects/project-list --project=demo --standalone --change-detection=OnPush --style=css --dry-run`
Service (`g s`)	`npx ng g s data-access/projects-api --project=demo --injectable --dry-run`
Direktive (`g d`)	`npx ng g d shared/highlight --project=demo --standalone --dry-run`
Pipe (`g p`)	`npx ng g p shared/display-label --project=demo --standalone --dry-run`
Interface (`g i`)	`npx ng g i domain/project --project=demo --dry-run`
Funktionaler Guard (`g g`)	`npx ng g g guards/auth --project=demo --functional --implements=CanActivate --dry-run`
Funktionaler Interceptor	`npx ng g interceptor http/correlation --project=demo --functional --dry-run`
Funktionaler Resolver (`g r`)	`npx ng g r data/project --project=demo --functional --dry-run`

Ein generierter Service ist noch kein API-Client mit Endpunkten. Ein generierter Guard enthält noch keine fachliche Berechtigungslogik. Ein Interceptor muss bewusst registriert und ein Resolver beziehungsweise Guard in den Routes verwendet werden. Generatoren liefern das Gerüst, nicht die Fachfunktion.

Für ein **neues, separates CLI-Projekt**, außerhalb eines vorhandenen Workspaces:

```
npx @angular/cli@22.2.3 new demo --standalone --routing --style=css
```

Das erstellt einen neuen Workspace und kann Abhängigkeiten installieren. Es wandelt dieses bestehende Nx-Repository nicht um.

Häufige Befehle gegenübergestellt

Aufgabe	In diesem Repository	Klassischer Angular-Workspace
Versionen	`npx nx report`	`npx ng version`
Projekte sehen	`npx nx show projects`	Projekte in `angular.json` ansehen
Projekt / Targets sehen	`npx nx show project ui`	`npx ng config projects`
Entwicklungsserver	`npm run storybook`; für eine echte App deren `serve`-Target	`npx ng serve demo`
Build	`npx nx build ui`	`npx ng build demo`
Unit-Tests	`npx nx test ui`	`npx ng test demo`
Storybook statisch	`npm run build-storybook`	Nur nach eigener Storybook-Einrichtung
Architekturbeispiele	`npm run test:architecture`	Kein allgemeiner Angular-CLI-Standardbefehl
Generatorhilfe	`npx nx g @nx/angular:component --help`	`npx ng g component --help`

`ui` ist hier eine Library und hat kein eigenes `serve`-Target. `storybook` ist der vorhandene interaktive Host. Für eine später generierte App zeigt `nx show project <name>`, welche Targets tatsächlich konfiguriert sind.

`ng add` installiert und konfiguriert ein Add-on; `ng update` plant bzw. führt Angular-Updates aus. In einem Nx-Workspace die passende Nx-Migration verwenden und alle betroffenen Pakete gemeinsam betrachten. Solche Befehle können Abhängigkeiten und Quellcode ändern und sind keine Diagnose- oder reinen Generatorvorschauen.

Und einen NgRx Signal Store anlegen?

`ng generate store` ist kein eingebauter Angular-CLI-Befehl. Auch das hier installierte `@ngrx/signals` 22.0.1 bringt in seiner Schematic-Collection nur die Initialisierung mit, keinen Signal-Store-Generator. Die Store-Datei im passenden Data-Access-Bereich wie im Kapitel **State und Signal Store** anlegen. Ein Service-Generator erzeugt nicht automatisch den dort beschriebenen Signal Store.

Ein kurzer Arbeitsablauf

Passenden Bereich wählen → kleine API festlegen → Generator mit `--dry-run` prüfen → wirklich generieren → Verhalten und Tokens ergänzen → öffentliches API prüfen → Story und sinnvolle Tests ergänzen → Build und Tests ausführen → Änderung auf einem Branch als Pull Request öffnen.

Komponenten und ihre APIs

Eine gute Aufteilung folgt der Verantwortung: Die Seite kennt den Ablauf, fachliche UI kennt ihre Darstellung, generische Controls kennen Interaktion und Gestaltung. Die Zahl der Komponenten allein ist kein Qualitätsmaß.

Die drei Ebenen

Ebene	Kennt	Meldet / tut
Generisches Control	Label, Wert, Disabled, Gestaltung	Kleine UI-Ereignisse und Formularbindung
Fachliche UI	Beispielsweise `Project` und verfügbare Darstellungsaktionen	Typisierte Ereignisse wie `publishRequested`
Feature-Page	Store, Routing, Formulardraft und Ablauf	Übersetzt Ereignisse in fachliche Methoden

Eine ProjectList braucht weder HttpClient noch einen eigenen Signal Store. Eine Feature-Page darf dagegen den passenden Store injizieren. Dependency Injection an der richtigen Grenze ist normal; fachliche UI muss nicht durch beliebig viele Ebenen mit Infrastruktur versorgt werden.

Eine kleine fachliche UI

```
import { ChangeDetectionStrategy, Component, input, output } from '@angular/core';
import { Button } from '@creaera/ui';
import { type Project } from './project.model';

@Component({
  selector: 'app-project-list',
  imports: [Button],
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `<ul>
    @for (project of projects(); track project.id) {
      <li>
        <button
          uiButton
          [variant]="selectedId() === project.id ? 'primary' : 'secondary'"
          [attr.aria-pressed]="selectedId() === project.id"
          (click)="selected.emit(project.id)"
        >
          {{ project.name }}
        </button>
        <span>{{ project.status }}</span>
        @if (project.status === 'draft') {
          <button uiButton [disabled]="busy()" (click)="publishRequested.emit(project)">
            Veröffentlichen
          </button>
```

```
        }
      </li>
    } @empty {
      <li>Keine Projekte gefunden.</li>
    }
  </ul>`,
})
export class ProjectList {
  readonly projects = input.required<readonly Project[]>();
  readonly selectedId = input<string | null>(null);
  readonly busy = input(false);
  readonly selected = output<string>();
  readonly publishRequested = output<Project>();
}
```

Die API beschreibt Absichten. `publishRequested` sagt mehr als `buttonClicked`. Die UI verändert die übergebenen Projekte nicht. Readonly-Daten und stabile IDs unterstützen einen klaren Datenfluss; `@for (...; track project.id)` bewahrt Elementidentitäten beim Aktualisieren der Liste.

Inputs nur für tatsächlich unterschiedliche Verwendungsszenarien ergänzen. Farben und Größen weiterhin über die zentralen Design-Tokens steuern. Ein Config-Objekt mit dutzenden optionalen Eigenschaften macht eine kleine Komponente nicht automatisch wiederverwendbar.

Die Feature-Page setzt den Ablauf zusammen

```
import { ChangeDetectionStrategy, Component, effect, inject } from '@angular/core';
import { FormControl, FormGroup, ReactiveFormsModule, Validators } from '@angular/forms';
import { Button } from '@creaera/ui';
import { ProjectList } from './project-list';
import { type CreateProject, type Project } from './project.model';
import { ProjectsStore } from './projects-store';

@Component({
  selector: 'app-project-page',
  imports: [ProjectList, Button, ReactiveFormsModule],
  providers: [ProjectsStore],
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `<h1>Projekte</h1>
    <label for="project-search">Suchen</label>
    <input id="project-search" type="search" [value]="store.query()" (input)="search($event)" />
    @if (store.loading()) {
      <p role="status">Projekte werden geladen.</p>
    } @else if (store.loadError(); as error) {
      <p role="alert">{{ error.message }}</p>
      <button uiButton (click)="store.reload()">Erneut laden</button>
    } @else {
      <app-project-list
        [projects]="store.entities()"
        [selectedId]="store.selectedId()"
        [busy]="store.writing()"
        (selected)="store.select($event)"
        (publishRequested)="publish($event)"
      />
    }
    <form [formGroup]="form" (ngSubmit)="create()">
      <label for="project-name">Projektname</label>
      <input id="project-name" formControlName="name" />
```

```
      @if (form.controls.name.invalid && form.controls.name.touched) {
        <p>Bitte einen Namen eingeben.</p>
      }
      <button uiButton type="submit" [disabled]="store.writing()">Anlegen</button>
    </form>
    @if (store.writeError(); as error) {
      <p role="alert">{{ error.message }}</p>
    }`,
})
export class ProjectPage {
  readonly store = inject(ProjectsStore);
  readonly form = new FormGroup({
    name: new FormControl('', { nonNullable: true, validators: [Validators.required] }),
  });

  private createAttempt: { input: CreateProject; requestId: string } | null = null;
  private readonly publishAttempts = new Map<string, { version: number; requestId: string }>();

  constructor() {
    this.store.reload();
    let completed = 0;
    // A UI side effect: clear the draft only after a confirmed server creation.
    effect(() => {
      const current = this.store.completedCreates();
      if (current > completed) {
        if (this.createAttempt?.input.name === this.form.controls.name.value) this.form.reset();
        this.createAttempt = null;
        completed = current;
      }
    });
  }
  search(event: Event): void {
    this.store.search((event.target as HTMLInputElement).value);
  }
  create(): void {
    this.form.markAllAsTouched();
    if (!this.form.valid || this.store.writing()) return;
    const input = this.form.getRawValue();
    if (!this.createAttempt || this.createAttempt.input.name !== input.name) {
      this.createAttempt = { input, requestId: crypto.randomUUID() };
    }
    this.store.create(this.createAttempt.input, this.createAttempt.requestId);
  }
  publish(project: Project): void {
    let attempt = this.publishAttempts.get(project.id);
    if (!attempt || attempt.version !== project.version) {
      attempt = { version: project.version, requestId: crypto.randomUUID() };
      this.publishAttempts.set(project.id, attempt);
    }
    this.store.publish(project, attempt.requestId);
  }
}
```

Der Store ist über `providers` an diese Seiteninstanz gebunden. Die Page koordiniert Suche, Auswahl, Formular und Veröffentlichung. Das Beispiel verwendet native Formularelemente, damit die Architektur unabhängig von den noch separat entwickelten Formularfeld-Komponenten verständlich bleibt. Diese können in einer konsumierenden Anwendung die nativen Wrapper ersetzen.

Das Formular wird erst nach einer bestätigten **Erstellung** zurückgesetzt. Ein fehlgeschlagener Request oder eine andere Aktion wie Veröffentlichen soll einen begonnenen Formulardraft erhalten. Auch ein während der Anfrage bereits geänderter Entwurf bleibt erhalten; zurückgesetzt wird nur die unveränderte abgesendete Eingabe. Bei größeren Formularen gehört dazu ein passender DTO- oder Draft-Versionsvergleich. Der Zähler im Beispiel ist eine kleine Erfolgsbestätigung für ein einzelnes Formular; bei mehreren Formularen pro Feature sind korrelierte Operationsergebnisse mit einer Request-ID geeigneter.

Lokales Formular oder Store-State?

Der noch nicht gespeicherte Entwurf gehört normalerweise in das Formular oder in lokalen Komponenten-State. Persistierte Serverdaten liegen im Feature-Store. Nicht jeden Tastendruck in eine globale Kopie des gesamten Datensatzes spiegeln.

Bei mehrstufigen Formularen, Draft-Wiederherstellung oder mehreren beteiligten Seiten kann ein eigener, bewusst begrenzter Draft-Store sinnvoll sein. Die fachliche Anforderung bestimmt die Lebensdauer.

Wann eine Komponente teilen?

Eine Aufteilung hilft, wenn ein Teil eine eigene Aufgabe, eine verständliche API, eigenständige Interaktion oder echte Wiederverwendung hat. Beispiele: Suchleiste, Ergebnisliste, Editor und Bestätigungsdialog.

Wenn das Template nur durch viele Weiterleitungen von Inputs/Outputs länger wird, ist eine weitere Ebene möglicherweise nicht hilfreich. Fachliche Container dürfen überschaubare Ablauflogik enthalten. Reine Berechnungen können in `computed` oder in kleine, getestete Domain-Funktionen wandern.

Signals passend einsetzen

- `input()` für Daten von außen, `output()` für typisierte Absichten.
- `model()` für ein tatsächlich gewolltes Two-Way-Value-API eines Controls.
- `signal()` für lokalen veränderbaren Zustand.
- `computed()` für abgeleitete Werte.
- `effect()` für echte Seiteneffekte, etwa das Zurücksetzen eines Formulars nach bestätigtem Erfolg.

Abgeleitete Zustände brauchen keine zweite manuell synchronisierte Kopie. Angulars Signal-Tracking und eine klare OnPush-Grenze passen zu diesem Ansatz. Reactive Forms bleiben im Beispiel die explizit getestete Formularlösung.

State und NgRx Signal Store

Signal Store ist eine Möglichkeit, Feature-Zustand, abgeleitete Werte und fachliche Methoden zusammenzufassen. Lokale Signals, die URL und der Server behalten dabei ihre eigenen Aufgaben.

Wer besitzt welchen Zustand?

Zustand	Typischer Besitzer
Offenes Menü, Hover, temporäre Ansicht	Lokale UI-Komponente
Noch nicht gespeicherter Formulardraft	Formular / lokaler Draft-State
Teilbare Suche, Filter, Sortierung, Seitenposition	URL, wenn Deep Links / Zurück-Navigation gewünscht sind
Geladene Projektdaten, Lade-/Fehlerzustände, laufende Aktionen	Feature-Store
Persistierte Wahrheit und erlaubte Zustandswechsel	Server
Tatsächlich app-weite Sitzung oder Präferenzen	Bewusst app-weiter Store

Im Referenzbeispiel ist die Suche bewusst lokal im Feature-Store. Für eine teilbare Suchseite wird sie stattdessen aus Query-Parametern angestoßen. Dann darf nicht zusätzlich eine unkoordinierte zweite Suchwert-Kopie entstehen.

Store-Lebensdauer

`ProjectsStore` hat kein `providedIn: 'root'`. Die Page stellt ihn bereit, sodass zwei Seiteninstanzen getrennten Zustand besitzen. Der getestete Lifecycle beendet auch die aktiven RxJS-Reads beim Zerstören des Injectors.

Für zusammengehörige Kindseiten kann eine Feature-Shell die Instanz teilen. Root-State nur dann verwenden, wenn die gewünschte Lebensdauer wirklich die App ist; bei einem Benutzerwechsel müssen benutzerabhängige Caches bewusst zurückgesetzt werden. Auf dem Server keine benutzerbezogenen Daten in prozessweiten, veränderbaren Modulvariablen halten.

Modell und DTO auseinanderhalten

```
export interface Project {
  readonly id: string;
  readonly name: string;
  readonly status: 'draft' | 'published';
  readonly version: number;
```

```
  readonly updatedAt: string;
}

export interface CreateProject {
  readonly name: string;
}
export interface UpdateProject {
  readonly name: string;
}

export interface ProjectDto {
  readonly id: string;
  readonly title: string;
  readonly status: 'draft' | 'published';
  readonly version: number;
  readonly updated_at: string;
}

export interface ProjectPageDto {
  readonly items: readonly ProjectDto[];
  readonly nextCursor: string | null;
}
export interface ProjectPage {
  readonly items: readonly Project[];
  readonly nextCursor: string | null;
}

export function fromProjectDto(dto: ProjectDto): Project {
  return {
    id: dto.id,
    name: dto.title,
    status: dto.status,
    version: dto.version,
    updatedAt: dto.updated_at,
  };
}
```

Das Domain-Modell verwendet `name` und `updatedAt`, der Beispielserver `title` und `updated_at`. Ein zentraler Mapper hält solche Unterschiede aus den Komponenten heraus. Das Domain-Modell verwendet hier serialisierbare Werte; Datumsformatierung bleibt eine Darstellungsaufgabe.

Der vollständige Store

```
import { computed, inject } from '@angular/core';
import { patchState, signalStore, withComputed, withMethods, withState } from '@ngrx/signals';
import { setAllEntities, withEntities } from '@ngrx/signals/entities';
import { rxMethod } from '@ngrx/signals/rxjs-interop';
import { tapResponse } from '@ngrx/operators';
import { defer, exhaustMap, pipe, switchMap, timer, type Observable } from 'rxjs';
import { ProjectsApi } from './projects-api';
import { type CreateProject, type Project, type UpdateProject } from './project.model';
import { toUserError, type UserError } from './user-error';

type ProjectCommand =
  | { kind: 'create'; input: CreateProject; requestId: string }
  | { kind: 'update'; project: Project; input: UpdateProject }
  | { kind: 'remove'; project: Project }
  | { kind: 'publish'; project: Project; requestId: string };
```

```
function execute(api: ProjectsApi, command: ProjectCommand): Observable<unknown> {
  switch (command.kind) {
    case 'create':
      return api.create(command.input, command.requestId);
    case 'update':
      return api.update(command.project, command.input);
    case 'remove':
      return api.remove(command.project);
    case 'publish':
      return api.publish(command.project, command.requestId);
  }
}

export const ProjectsStore = signalStore(
  withState(() => ({
    query: '',
    selectedId: null as string | null,
    loading: false,
    writing: false,
    loadError: null as UserError | null,
    writeError: null as UserError | null,
    completedCreates: 0,
  })),
  withEntities<Project>(),
  withComputed(({ entityMap, selectedId }) => ({
    selectedProject: computed(() => entityMap()[selectedId() ?? ''] ?? null),
  })),
  withMethods((store, api = inject(ProjectsApi)) => {
    const load = rxMethod<{ query: string; delay: number }>(
      pipe(
        // Cancel both the old debounce timer and any old HTTP read immediately.
        switchMap(({ query, delay }) =>
          defer(() => {
            patchState(store, { loading: true, loadError: null });
            return timer(delay).pipe(
              switchMap(() => api.list(query)),
              tapResponse({
                next: (page) => patchState(store, setAllEntities([...page.items])),
                error: (error) => patchState(store, { loadError: toUserError(error) }),
                finalize: () => patchState(store, { loading: false }),
              }),
            );
          }),
        ),
      ),
    );
    const write = rxMethod<ProjectCommand>(
      pipe(
        // This feature accepts one write at a time; repeated clicks are ignored.
        exhaustMap((command) =>
          defer(() => {
            patchState(store, { writing: true, writeError: null });
            return execute(api, command).pipe(
              tapResponse({
                next: () => {
                  patchState(store, {
                    completedCreates:
                      store.completedCreates() + (command.kind === 'create' ? 1 : 0),
                  });
                  // Re-query the current server filter instead of guessing list membership.
                  load({ query: store.query(), delay: 0 });
                },
```

```
                error: (error) => patchState(store, { writeError: toUserError(error) }),
                finalize: () => patchState(store, { writing: false }),
              }),
            );
          }),
        ),
      ),
    );
    return {
      reload(): void {
        load({ query: store.query(), delay: 0 });
      },
      search(query: string): void {
        if (query === store.query()) return;
        patchState(store, { query });
        load({ query, delay: 250 });
      },
      select(id: string | null): void {
        patchState(store, { selectedId: id });
      },
      create(input: CreateProject, requestId: string): void {
        write({ kind: 'create', input, requestId });
      },
      update(project: Project, input: UpdateProject): void {
        write({ kind: 'update', project, input });
      },
      remove(project: Project): void {
        write({ kind: 'remove', project });
      },
      publish(project: Project, requestId: string): void {
        write({ kind: 'publish', project, requestId });
      },
    };
  }),
);
```

Die Bausteine verstehen

API	Aufgabe im Beispiel
`signalStore`	Erzeugt den injizierbaren Store aus Features
`withState`	Query, Auswahl-ID, getrennte Read-/Write-Zustände
`withEntities<Project>()`	Normalisierte `entityMap`, `ids` und abgeleitete `entities`
`withComputed`	`selectedProject` aus ID und Entities ableiten
`withMethods`	Kleine fachliche Methoden veröffentlichen
`patchState`	Zustandsänderungen kontrolliert und unveränderlich anwenden
`rxMethod`	RxJS-Nebenläufigkeit und Lifecycle an den Store binden
`tapResponse`	Erfolg, Fehler und Abschluss innerhalb einer Anfrage behandeln

State-Features stehen vor Features, die darauf zugreifen. Die State-Factory liefert frische Startwerte je Instanz. Die intern erzeugten Read-/Write-Streams werden nicht als öffentliche generische Execute-API veröffentlicht.

`withProps` ist sinnvoll, wenn mehrere Store-Features dieselbe Abhängigkeit oder abgeleitete Eigenschaft brauchen. Hier reicht die Injection im Methoden-Feature. `withHooks` kann einen immer benötigten Initial-Load starten; das Beispiel lässt die Page ausdrücklich `reload()` aufrufen. Neue APIs wie Resource-Integration oder Events gezielt nach Versionsstand und Anforderungen einsetzen.

Wichtige Entscheidungen des Beispiels

Die Entities repräsentieren **die aktuelle erste Ergebnisseite**, keinen vollständigen globalen Projektcache. Die API liefert einen Cursor, aber ein „Weitere laden“-Ablauf ist hier noch nicht implementiert. Bei Filterwechseln wird die Liste vom Server ersetzt. Eine getrennte Detailansicht benötigt einen eigenen klaren Cache-/Ladevertrag.

`selectedProject` ist abgeleitet. Wird eine gewählte ID herausgefiltert, ist dieser Wert null, obwohl die Auswahl-ID zunächst bestehen bleibt. Die Anwendung entscheidet, ob sie die Auswahl in diesem Fall explizit zurücksetzt.

Nach Mutationen lädt der Store die aktuelle Liste neu. Das hält Serverfilter und Sortierung konsistent. Blindes Einfügen einer neuen Entity würde bei einer gefilterten oder paginierten Liste möglicherweise einen falschen Inhalt zeigen.

Keine Promise-Annahme bei rxMethod

Die veröffentlichten Command-Methoden im Beispiel geben `void` zurück. `await store.create(...)` würde hier nicht auf HTTP-Erfolg warten. Die UI liest `writing`, Fehler und die bestätigten Ergebnisse. Eine alternative, bewusst als Promise entworfene Methode ist möglich, braucht aber denselben klaren Vertrag für Fehler, Nebenläufigkeit und Lifecycle.

Der Store hält eine parallele Leseoperation und maximal einen Schreibvorgang. Die passenden Operatoren und die Gründe dafür stehen im Kapitel **Aktionen und Nebenläufigkeit**.

HTTP und CRUD-APIs

Eine gute API beschreibt Ressourcen und fachliche Verträge. Die Oberfläche bekommt typisierte Methoden; URL-Bau, Header und DTO-Mapping liegen am HTTP-Rand. Der Server entscheidet über Validierung, Berechtigungen und zulässige Zustandsänderungen.

Ein konkreter Ressourcenvertrag

Anfrage	Erfolg	Bedeutung
`GET /api/projects?q=ui&limit=20&cursor=...`	200	Gefilterte Ergebnisseite
`GET /api/projects/{id}`	200	Einzelnes Projekt
`POST /api/projects`	201 + Location	Projekt anlegen
`PATCH /api/projects/{id}`	200	Erlaubte Felder ändern
`DELETE /api/projects/{id}`	204	Projekt löschen
`POST /api/projects/{id}/actions/publish`	200	Fachlichen Zustandswechsel ausführen

Die Referenz implementiert den Client gegen diesen Beispielvertrag. Ein Backend ist nicht Teil des Repositories. `PATCH` akzeptiert hier nur den Namen; der Statuswechsel wird über eine fachliche Aktion ausgeführt.

Liste und Pagination

```
{
  "items": [
    {
      "id": "p1",
      "title": "Designsystem",
      "status": "draft",
      "version": 3,
      "updated_at": "2026-10-24T10:00:00Z"
    }
  ],
  "nextCursor": "opaque-next-page-token"
}
```

`nextCursor: null` bedeutet letzte Seite. Der Server begrenzt `limit`, definiert eine stabile Reihenfolge mit einem eindeutigen Tie-Breaker wie der ID und bindet Cursor an die passenden Filter. Clients interpretieren oder erzeugen Cursor nicht selbst. Bei Filterwechsel beginnt die Pagination erneut.

Das Beispiel fragt die erste Seite mit Limit 20 ab. Weitere Seiten werden im Client-Vertrag unterstützt, aber noch nicht im Store angehängt. Für einen vollständigen Pagination-Ablauf gehören Cursor, Ladezustände und Deduplizierung bewusst zum Feature.

Typisierter API-Client

```
import { HttpClient, HttpHeaders, HttpParams } from '@angular/common/http';
import { Injectable, InjectionToken, inject } from '@angular/core';
import { map, type Observable } from 'rxjs';
import {
  fromProjectDto,
  type CreateProject,
  type Project,
  type ProjectDto,
  type ProjectPage,
  type ProjectPageDto,
  type UpdateProject,
} from './project.model';

export const PROJECTS_API_URL = new InjectionToken<string>('PROJECTS_API_URL', {
  factory: () => '/api/projects',
});

@Injectable({ providedIn: 'root' })
export class ProjectsApi {
  private readonly http = inject(HttpClient);
  private readonly base = inject(PROJECTS_API_URL);
  private url(id: string): string {
    return `${this.base}/${encodeURIComponent(id)}`;
  }
  private version(version: number): HttpHeaders {
    // This example backend defines an ETag as the quoted DTO version.
    return new HttpHeaders({ 'If-Match': `"${version}"` });
  }
  list(query: string, cursor?: string): Observable<ProjectPage> {
    let params = new HttpParams().set('q', query.trim()).set('limit', 20);
    if (cursor) params = params.set('cursor', cursor);
    return this.http
      .get<ProjectPageDto>(this.base, { params })
      .pipe(
        map((page) => ({ items: page.items.map(fromProjectDto), nextCursor: page.nextCursor })),
      );
  }
  get(id: string): Observable<Project> {
    return this.http.get<ProjectDto>(this.url(id)).pipe(map(fromProjectDto));
  }
  create(input: CreateProject, requestId: string): Observable<Project> {
    return this.http
      .post<ProjectDto>(
        this.base,
        { title: input.name },
        {
          headers: new HttpHeaders({ 'Idempotency-Key': requestId }),
        },
      )
      .pipe(map(fromProjectDto));
  }
  update(project: Project, input: UpdateProject): Observable<Project> {
    return this.http
```

```
      .patch<ProjectDto>(
        this.url(project.id),
        { title: input.name },
        {
          headers: this.version(project.version),
        },
      )
      .pipe(map(fromProjectDto));
  }
  remove(project: Project): Observable<void> {
    return this.http.delete<void>(this.url(project.id), { headers: this.version(project.version) });
  }
  publish(project: Project, requestId: string): Observable<Project> {
    return this.http
      .post<ProjectDto>(
        `${this.url(project.id)}/actions/publish`,
        {},
        {
          headers: this.version(project.version).set('Idempotency-Key', requestId),
        },
      )
      .pipe(map(fromProjectDto));
  }
}
```

Der API-Client ist zustandsarm und kann deshalb app-weit bereitgestellt werden. Ladezustände gehören zum jeweiligen Verbraucher. Es gibt hier keine globale Variable `isLoading` für sämtliche HTTP-Anfragen.

`HttpParams` übernimmt das Encoding. IDs werden für Pfadsegmente kodiert. Create- und Update-DTOs erlauben nur die vom Server vorgesehenen Eingaben; `Partial<Project>` würde etwa ID, Status und Version ungewollt zu Schreibfeldern machen. Für unterschiedliche Anwendungsfälle sind explizite Methoden gut lesbar.

`HttpClient<T>` prüft die Antwort **nicht zur Laufzeit**. Für unzuverlässige oder kritische Grenzen gehört eine Schema-Prüfung vor das DTO-Mapping. Das Beispiel setzt einen vereinbarten, getesteten Antwortvertrag voraus.

Gleichzeitige Änderungen erkennen

Ein Client sendet beim Ändern oder Löschen `If-Match`. Der Beispielserver definiert seinen ETag als in Anführungszeichen gesetzte DTO-Version, zum Beispiel `"3"`. In einer echten API sind ETags häufig opake Strings; dann den vom Server empfangenen ETag unverändert halten und zurücksenden.

Bei veralteter Version liefert der Server **412 Precondition Failed**. Die UI bietet Neuladen beziehungsweise einen bewussten Konfliktablauf an. Eine alte Version wird nicht still durch die gerade geladene ersetzt und erneut gesendet, weil dadurch fremde Änderungen überschrieben werden könnten.

Einheitliche Fehler

Für API-Fehler eignet sich `application/problem+json` nach RFC 9457. Ein Beispiel mit einer dokumentierten Erweiterung für Feldfehler:

```
{
  "type": "https://api.example.com/problems/validation",
  "title": "Eingaben ungültig",
  "status": 422,
  "instance": "/requests/req-123",
  "errors": { "name": ["Der Name darf höchstens 120 Zeichen lang sein."] }
}
```

`errors` ist eine eigene Erweiterung, kein automatisch festgelegter Teil des Standards. Der Vertrag muss festlegen, wie Serverfelder zu Formularfeldern passen. Der Demo-Client zeigt zunächst allgemeine, kontrollierte Meldungen:

```
import { HttpErrorResponse } from '@angular/common/http';

export interface UserError {
  readonly message: string;
  readonly status: number | null;
}

export function toUserError(error: unknown): UserError {
  const status = error instanceof HttpErrorResponse ? error.status : null;
  const message =
    status === 0
      ? 'Die Verbindung ist unterbrochen.'
      : status === 412
        ? 'Die Daten wurden inzwischen geändert. Bitte neu laden.'
        : status === 409
          ? 'Die Aktion ist im aktuellen Zustand nicht möglich.'
          : status === 422
            ? 'Bitte die Eingaben prüfen.'
            : status === 401
              ? 'Bitte erneut anmelden.'
              : status === 403
                ? 'Für diese Aktion fehlt die Berechtigung.'
                : 'Die Anfrage konnte nicht abgeschlossen werden.';
  return { message, status };
}
```

Status	Typische UI-Reaktion
0 im Angular-Client	Netzwerk-/Verbindungsfehler anzeigen
401	Sitzung / Anmeldung klären
403	Fehlende Berechtigung erklären
404	Nicht mehr vorhandene Ressource behandeln
409	Fachlicher Konflikt / Zustand passt nicht
412	Veraltete Version, bewusster Reload-/Konfliktablauf
422	Eingaben korrigieren, Feldfehler zuordnen
429	Wartezeit / Retry-After berücksichtigen
5xx	Allgemeine Störung, passende Wiederholungsoption anbieten

Der kleine Error-Mapper deckt nicht jeden Status gesondert ab. Read- und Write-Fehler bleiben im Store getrennt. Backend-Details und Stacktraces gehören nicht ungeprüft in eine Meldung der Oberfläche.

Infrastruktur an der richtigen Stelle

Basis-URLs über Konfiguration oder einen InjectionToken bereitstellen. Auth-/Korrelations-Header können funktionale HTTP-Interceptors übernehmen. Die gewählte Session-Lösung bestimmt Cookie-, Token- und XSRF-Verhalten; Berechtigungen werden auf dem Server geprüft.

Eine geänderte Domäne erfordert gegebenenfalls einen passenden CORS-Vertrag für Header wie `If-Match` und `Idempotency-Key`. Fachliche Fehlerbehandlung bleibt beim Feature. Ein globaler Interceptor sollte nicht unbemerkt alle Writes wiederholen oder jede fachliche Meldung als denselben Toast anzeigen.

Aktionen und Nebenläufigkeit

CRUD beschreibt Änderungen an Ressourcen. Manche Vorgänge haben zusätzliche Geschäftsregeln: veröffentlichen, genehmigen, archivieren oder eine Berechnung starten. Eine passende API gibt solchen Absichten einen klaren Vertrag.

Ein expliziter fachlicher Command

```
POST /api/projects/p1/actions/publish
Content-Type: application/json
If-Match: "3"
Idempotency-Key: 0b77a144-4e94-4d26-a755-000000000001

{}
```

Der Server prüft Berechtigung, Version und den erlaubten Zustandswechsel. Die Antwort enthält im Beispiel das aktualisierte Projekt. Ein beliebiges `PATCH { "status": "published" }` würde diese Absicht und ihre Voraussetzungen weniger ausdrücklich darstellen.

Der Pfad `/actions/publish` ist eine hier gewählte Konvention. Manche APIs modellieren denselben Vorgang als fachliche Unterressource, etwa eine Veröffentlichung. Wichtig sind konsistente Semantik, Vorbedingungen und ein nachvollziehbares Ergebnis, nicht eine einzige verbindliche URL-Schreibweise.

In der UI lautet das Ereignis `publishRequested`; am Store und API-Client heißt die Methode `publish`. So bleibt die fachliche Absicht über die Schichten lesbar.

Den passenden RxJS-Operator wählen

Operator	Geeignetes Muster	Verhalten
`switchMap`	Suche / neueste Lesesicht	Vorherige Subscription beenden, neueste Sicht übernehmen
`exhaustMap`	Ein Schreibvorgang, Doppelklick vermeiden	Weitere Auslöser während des laufenden Vorgangs ignorieren
`concatMap`	Gewollte Reihenfolge von Commands	Jeder Auslöser wird nacheinander ausgeführt
`mergeMap`	Unabhängige parallele Vorgänge	Mehrere Requests gleichzeitig; Zustand je Vorgang verwalten

Der Referenz-Store verwendet `switchMap` für Reads und einen gemeinsamen `exhaustMap` für alle Writes dieses Features. Das bedeutet bewusst: Während irgendein Write läuft, wird ein zweiter Write ignoriert. Die UI sperrt die entsprechenden Aktionen zusätzlich mit `writing`.

Für einen großen Tabelleneditor können unabhängige Vorgänge je Datensatz angemessener sein. Dann gehören Pending-/Fehlerzustände und Konkurrenzkontrolle zu den jeweiligen IDs. Ein einzelnes globales Boolean reicht bei echten parallelen Requests nicht.

Client-Abbruch ist kein Server-Rollback

Eine beendete HTTP-Subscription kann den Client-Request abbrechen. Der Server kann einen Schreibvorgang zu diesem Zeitpunkt bereits verarbeitet haben. Deshalb ist `switchMap` keine allgemeine Rückgängig-Funktion für Mutationen. Ein fachlicher Abbruch benötigt einen eigenen Serververtrag.

Idempotenz bewusst entwerfen

Create und Publish erhalten im Beispiel eine Request-ID. Die Page merkt sich beim Anlegen Payload und Key. Ein weiterer Versuch mit unverändertem Payload verwendet denselben Key; erst eine andere Eingabe oder ein bestätigter Erfolg beendet diesen Versuch. Für Publish wird der Key pro Projektversion gehalten. Der HTTP-Client erzeugt nicht bei jeder Wiederholung einen neuen Key.

Der Server muss Idempotenz tatsächlich implementieren, etwa über den Schlüssel aus Akteur, Operation und Idempotency-Key mit einem Payload-Hash und gespeichertem Ergebnis. Gleichzeitig eintreffende gleiche Keys dürfen nicht beide die Wirkung ausführen. Derselbe Key mit anderem Payload ist ein Konflikt. Aufbewahrungsdauer und Wiederholungsverhalten gehören in den API-Vertrag.

Eine Wiederholung **derselben** fehlgeschlagenen Absicht muss ihren Key behalten. Automatische Write-Retries sind im Beispiel nicht eingebaut. Die gespeicherten Versuche leben nur so lange wie die Page. Für Wiederaufnahme nach einem Reload braucht das Produkt eine passende Persistenz- oder Operationsstatus-Lösung. Ein neuer Payload ist eine neue Absicht, keine unbemerkte Wiederholung der alten.

HTTP-Idempotenz beschreibt die beabsichtigte Wirkung, nicht zwingend identische Statuscodes. Ein wiederholtes DELETE kann je nach dokumentiertem Vertrag etwa 204 oder 404 liefern, obwohl die Ressource in beiden Fällen nicht mehr existiert.

Erfolg, Fehler und erneute Versuche

`tapResponse` sitzt im inneren HTTP-Stream. So beendet ein einzelner Fehler nicht die ganze `rxMethod`-Eingabepipeline. `finalize` räumt den jeweiligen Pending-Zustand auf. Ein Write-Fehler verändert im Beispiel keine Entities.

Nach Erfolg wird die aktuelle Serverliste neu angefragt. Eine lokale optimistische Aktualisierung ist möglich, benötigt aber einen konkreten Plan für Rücknahme, Versionen, Filterzugehörigkeit und Pagination. Für den Einstieg ist der bestätigte Serverzustand oft leichter korrekt zu halten.

Retries für Reads und Writes getrennt entscheiden. Für geeignete Reads können begrenzte Versuche mit Backoff und gegebenenfalls Retry-After passen. Writes benötigen zusätzlich klare Idempotenz-/Konfliktregeln. Der Beispiel-Client führt keine automatischen HTTP-Retries aus.

Lang laufende Aktionen

Für einen länger dauernden Auftrag ist dieser alternative Vertrag sinnvoll:

```
HTTP/1.1 202 Accepted
Location: /api/operations/op-123
Retry-After: 2
Content-Type: application/json

{ "id": "op-123", "status": "queued" }
```

Die UI verfolgt anschließend `/api/operations/op-123` per Polling oder einer vereinbarten Push-Verbindung. Der Operationszustand kann beispielsweise `queued`, `running`, `succeeded`, `failed` und `cancelled` umfassen. Eine angenommene Anfrage ist noch kein fachlich abgeschlossener Vorgang.

Polling endet bei einem terminalen Zustand oder beim passenden UI-Lifecycle. Ein Client-Timeout kann einen unbekannten Ausgang bedeuten; vor einem erneuten Command zunächst den Operations-/Idempotenzstatus klären. Dieser 202-Ablauf ist ein Architekturvertrag und nicht Teil des synchronen Publish-Referenzclients.

Server-Suche und Autocomplete

Bei einer Server-Suche besitzt die UI den Eingabetext, ein Feature-Store den Anfragezustand und ein API-Client den HTTP-Vertrag. Das generische Eingabefeld kennt weder URLs noch Fachregeln.

Die richtige Reihenfolge

```
Neuer Suchwert
  → alten Debounce-Timer und alten Read sofort beenden
  → kurze Pause vor dem neuen Request
  → API anfragen
  → nur Ergebnis der aktuellen Subscription übernehmen
```

Der vollständige Code steht im Kapitel **State und Signal Store**. Entscheidend ist die Struktur des dortigen `load`-Helpers:

```
rxMethod-Eingabe
  → switchMap
      → defer: loading einschalten
      → timer(delay)
      → switchMap: api.list(query)
      → tapResponse: Ergebnis / Fehler / finalize
```

Der Timer liegt innerhalb des äußeren `switchMap`. Dadurch wird ein älterer HTTP-Read schon beim neuen Suchwert beendet und nicht erst nach der neuen Debounce-Pause. Andernfalls könnte ein altes Ergebnis in dieser Pause noch die neue Ansicht überschreiben. Genau diese Reihenfolge wird im Referenztest geprüft.

Ladezustand ohne Rennen

Die vorherige Subscription wird zuerst beendet. Ihr `finalize` läuft, bevor `defer` der neuen Subscription `loading` erneut einschaltet. Dadurch kann der Abschluss des alten Requests den neuen Ladezustand nicht verspätet löschen.

Bei anderen Nebenläufigkeitsmodellen, etwa mehreren gleichzeitigen Reads, werden stattdessen requestbezogene IDs, Zähler oder Status-Maps benötigt. Nicht denselben Boolean aus beliebig vielen unabhängigen Subscriptions setzen.

Eingabetext und Serverparameter

Die Page übergibt den rohen Text an `store.search`. Der Store behält ihn als UI-Wert; der API-Client trimmt den Suchparameter für den Server. So wird der Benutzer nicht bei jedem Leerzeichen mitten beim Tippen korrigiert.

Der Demo-Store wartet 250ms auf eine neue Suche und lädt bei `reload()` ohne zusätzliche Suchpause. Die Dauer ist eine Startannahme und sollte zum Produkt passen. Fehler lassen die Eingabepipeline weiterleben, sodass eine neue Suche oder ein Reload danach wieder funktioniert.

Autocomplete mit einer Serverliste

Die bestehende Autocomplete-Komponente erhält derzeit Textvorschläge über `options` und filtert diese zusätzlich lokal. Eine Anwendung kann dieses Array asynchron ersetzen. Das ist noch kein dediziertes API für reine Server-Suche.

Für eine passende spätere Erweiterung sind drei kleine, klar getrennte Vertragsbestandteile sinnvoll:

Bestandteil	Verantwortung
Suchtext-Ereignis	UI meldet die Eingabe an das Feature
Ladezustand	UI zeigt, dass die aktuelle Suche läuft
Lokale Filterung abschaltbar	Server-Ranking, Synonyme oder Fuzzy-Ergebnisse bleiben erhalten

Diese Erweiterung wird hier beschrieben und nicht als bereits implementierte Autocomplete-API ausgegeben. Der Serverzugriff bleibt im API-Client und Store. Das UI-Control sollte nicht selbst eine URL oder ein HttpClient benötigen.

Wenn der Server anhand von Synonymen „München“ für „Munich“ findet, würde eine zusätzliche einfache lokale Teilstringsuche möglicherweise genau dieses Ergebnis entfernen. Deshalb muss die Verantwortung für die Filterung bewusst festgelegt werden.

Query und Auswahl getrennt halten

Bei fachlichen Vorschlägen wie Kunden oder Projekten ist häufig die stabile ID der gewählten Entity relevant, während der Suchtext nur deren sichtbares Label ist. Dafür braucht eine spätere Objekt-Autocomplete-API einen ausdrücklich definierten Wertvertrag. Freie Texteingabe, gewählte ID und angezeigter Text sollten dann nicht versehentlich denselben State-Slot verwenden.

Das aktuelle Autocomplete speichert einen String. Ein Formular kann einen eigenen Validator verwenden, wenn es nur existierende Vorschläge zulassen soll. Die Architekturbeispiele selbst verwenden die Suchliste der Projektverwaltung und setzen die noch offenen Formular-/Autocomplete-PRs nicht voraus.

URL und Lifecycle

Für eine teilbare Ergebnisseite können Query-Parameter die Suchquelle sein. Die UI aktualisiert dann die URL und der Store reagiert auf diese Quelle. Die Registrierung einer Signal-/Observable-Eingabe an einer `rxMethod` gehört in einen passenden Injection-/Lifecycle-Kontext und wird einmal eingerichtet, nicht erneut

bei jedem `effect`-Durchlauf.

Beim Verlassen der im Beispiel komponentenlokal bereitgestellten Seite werden offene Reads beendet. Ein Write-Abbruch macht dessen Serverwirkung nicht rückgängig; dafür gelten die Regeln aus **Aktionen und Nebenläufigkeit**.

Tests und schrittweise Weiterentwicklung

Architektur wird belastbar, wenn ihre wichtigsten Grenzen und Abläufe geprüft werden. Viele kleine Mocks allein beweisen noch keinen korrekten Datenfluss.

Ausführbare Referenz

Die Dateien unter `libs/storybook/examples/architecture` sind echte Angular- und NgRx-Quellen. Die Dokumentation übernimmt deren Codeblöcke automatisch. Sie lassen sich aus dem Repository-Root prüfen:

```
npm ci
npm run docs:architecture:check
npm run test:architecture
npm run build-storybook
```

`docs:architecture:check` erkennt Abweichungen zwischen den Quellen und ihren Codeblöcken. Mit `npm run docs:architecture:sync` werden die Blöcke nach einer bewussten Quellenänderung aktualisiert. Der eigene Test-Target verwendet Angulars Compiler und das HTTP-Testbackend. Es wird keine echte Backend-Verbindung vorausgesetzt und kein Netzwerk-End-to-End-Erfolg behauptet.

Was geprüft wird

Grenze / Ablauf	Aussage des Tests
API-Client	HTTP-Methode, Parameter, DTO-Mapping und Header stimmen
Read-Pipeline	Alter Read wird sofort storniert; nur der neue startet nach der Pause
Fehler	Ein HTTP-Fehler beendet nicht alle künftigen Read-/Write-Aufrufe
Write-Pipeline	Gleichzeitig ausgelöste Writes werden gemäß exhaustMap-Regel behandelt
Mutationserfolg	Aktueller Filter wird neu geladen, statt unpassende Entities einzufügen
Konflikt	412 wird sichtbar und verändert keine Daten optimistisch
Lifecycle	Zerstören des Injectors beendet offene Reads
Store-Scope	Zwei Pages erhalten getrennte Store-Instanzen

Grenze / Ablauf	Aussage des Tests
Formular	Publish erhält den Entwurf; erst eine bestätigte Erstellung setzt ihn zurück

Ein Test gegen einen Mock-Server ersetzt keine Backend-Vertragstests. Ergänzend braucht ein Produkt mindestens aussagekräftige Backend-Tests und repräsentative Browserabläufe gegen eine Testumgebung.

Welche Tests auf welche Ebene gehören

Reine Domain-Regeln können ohne Angular getestet werden. Der API-Client wird mit `provideHttpClientTesting()` gegen Request- und Response-Verträge geprüft. Store-Tests prüfen den öffentlichen Zustand und konkrete Abläufe einschließlich Fehlern, Stornierung und Konkurrenz. UI-Tests prüfen Beschriftung, Interaktion, Outputs, Fokus und Formulare.

Ein Browser-Smoke-Test verbindet die relevanten Teile: suchen, anlegen, Fehler korrigieren und eine fachliche Aktion ausführen. Dabei muss unterscheidbar sein, ob wirklich ein Backend aufgerufen wurde oder nur ein Testdouble geantwortet hat.

Anforderungsgetriebener Ausbau

- Mehrere Seiten brauchen denselben Zustand: eine passende Feature-Shell oder einen ausdrücklich gemeinsamen Cache einführen.
- Viele Entities: Pagination, serverseitige Suche oder Virtualisierung nach tatsächlichem Bedarf ergänzen; ein Entity-Store rendert nicht automatisch große Listen performant.
- Parallele Aktionen: Pending-/Fehlerstatus je Entity oder Operation modellieren.
- Mehrstufige Formulare: eigenen Draft-Lifecycle und bestätigte Speicherpunkte klären.
- Offline/optimistische UI: Konflikte, Rollback und Serverabgleich vorher entwerfen.
- Generische Store-Features: erst aus mehreren echten, gleichen Abläufen abstrahieren. Fachliche Methoden dürfen lesbar bleiben.

Gute öffentliche APIs erhalten

Neue Inputs und Methoden mit einem konkreten Consumer begründen. Eine kleine Komponenten-API kann sich später gezielt erweitern; das Entfernen eines breiten, bereits verwendeten Config-Objekts ist aufwendiger.

Für HTTP-Verträge sind kompatible Ergänzungen, versionierte Änderungen bei Brüchen und ein überprüfbares Schema sinnvoll. OpenAPI kann Clients und Vertragstests unterstützen; generierte Transporttypen müssen nicht automatisch das Domain-Modell jeder UI werden.

Eine Anwendung sollte dokumentieren, wer welche Daten besitzt, wie lange eine Instanz lebt und was nach Fehler, Erfolg, Reload und Benutzerwechsel passiert. Diese Entscheidungen helfen später mehr als eine große Zahl abstrakter Basisklassen.